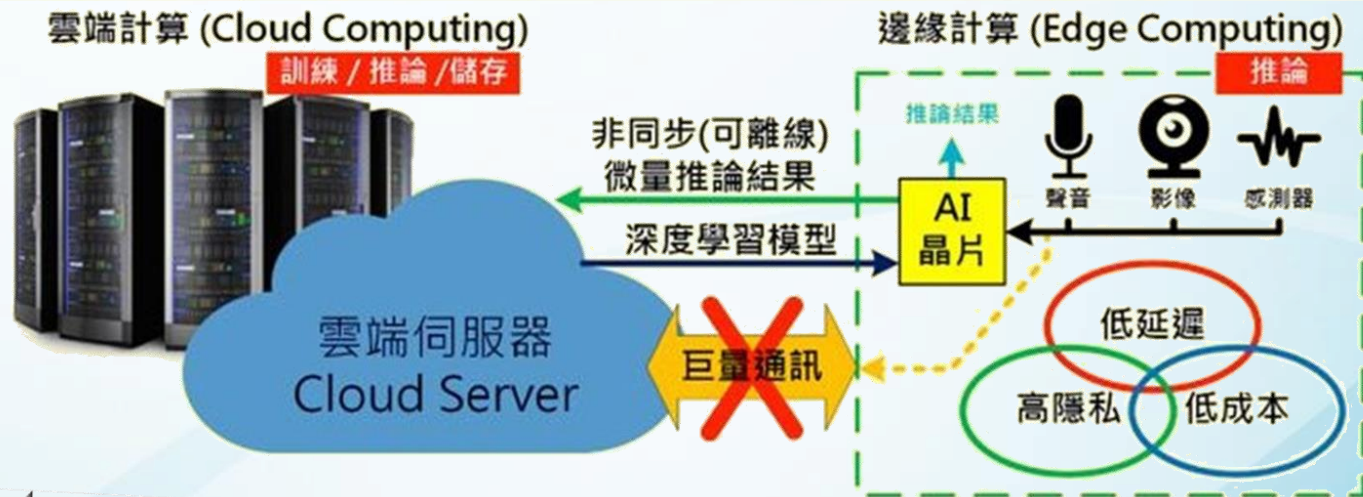
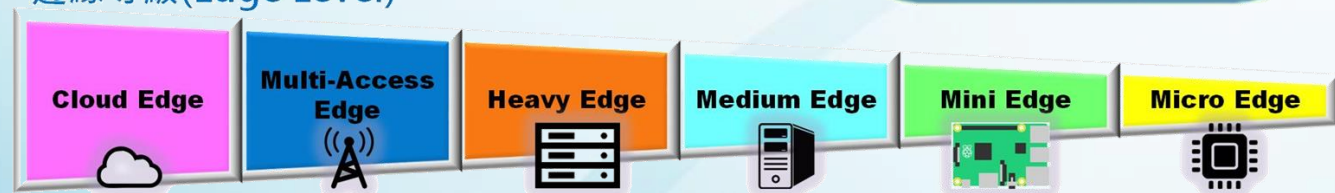


邊緣人工智慧實務 (EE5354701)



Edge AI

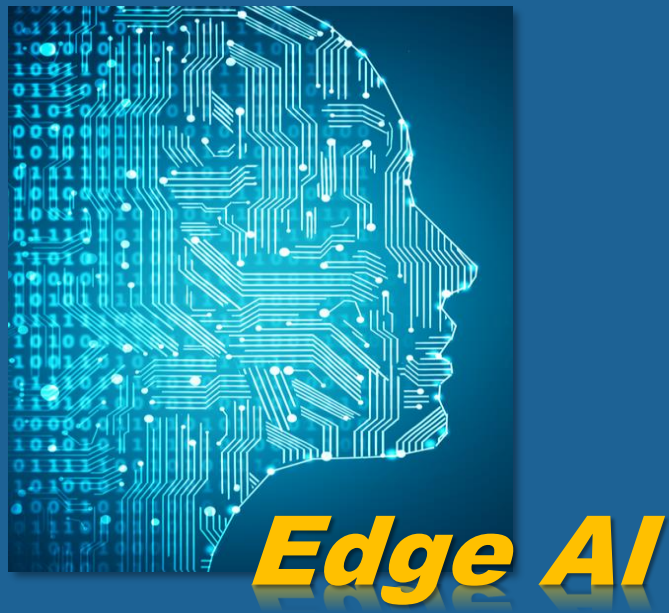
邊緣等級(Edge Level)



5 模型優化與部署

電機工程系 人工智慧應用產碩專班 許哲豪 兼任助理教授

簡報大綱

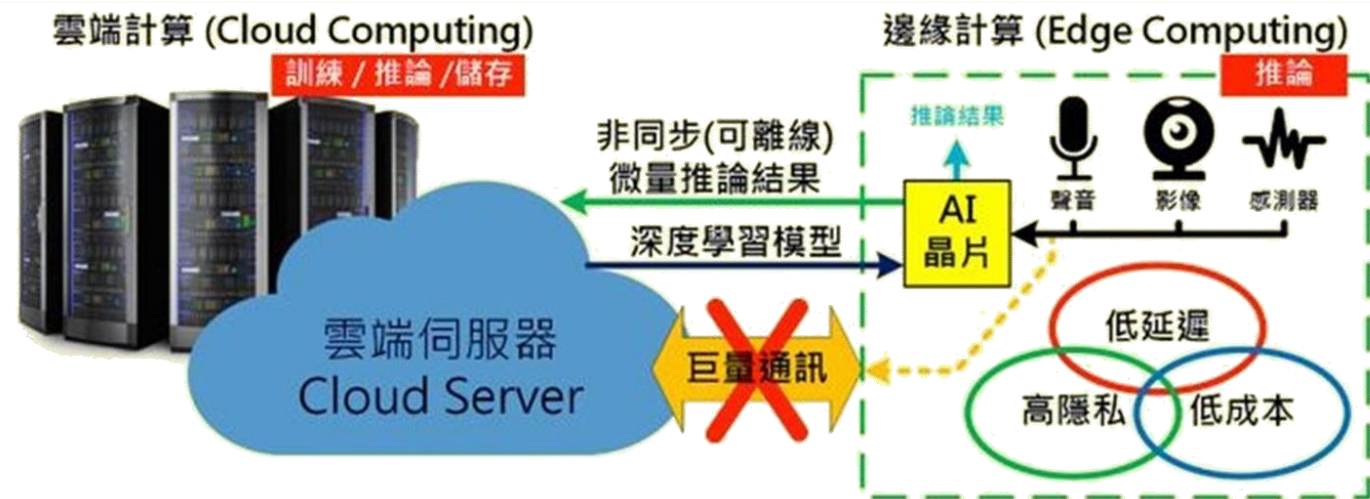


- 5.1 模型訓練優化
- 5.2 加速訓練方式
- 5.3 模型推論優化

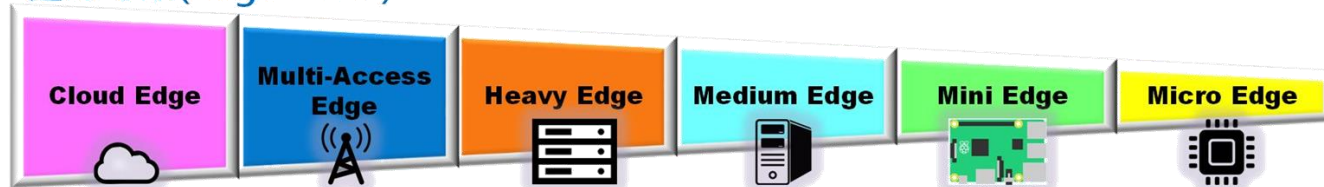
https://hackmd.io/@OmniXRI-Jack/NTUST_EdgeAI_2026



Edge AI



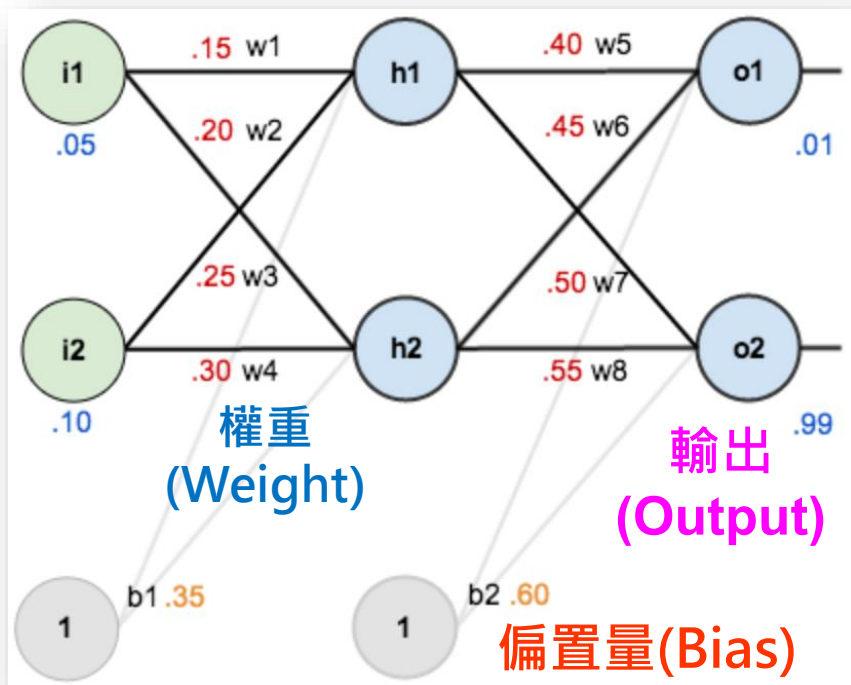
邊緣等級(Edge Level)



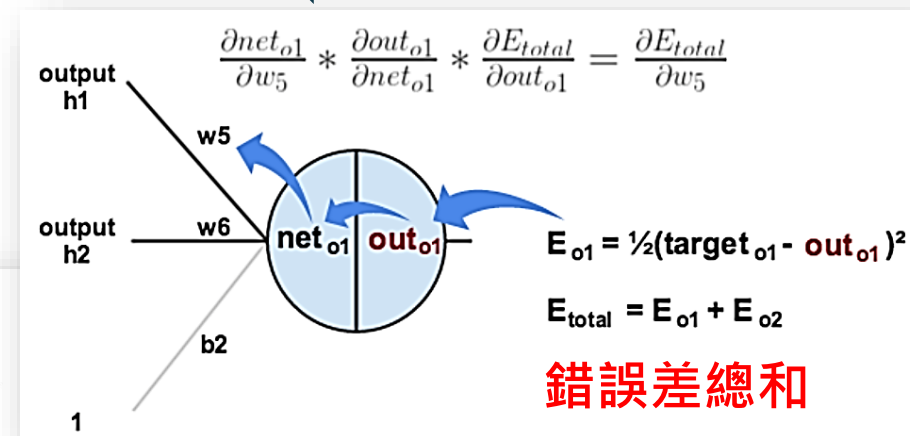
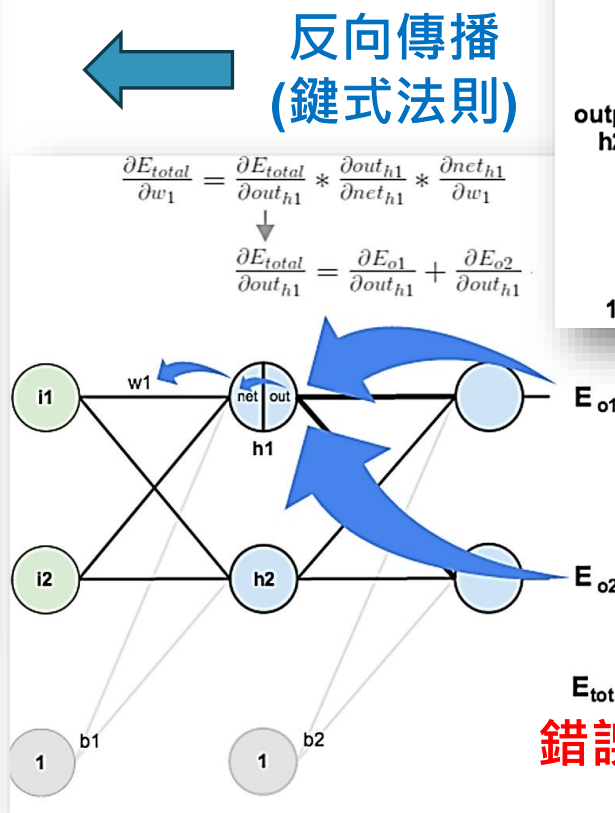
5.1 模型訓練優化

神經網路訓練與推論

正向推論 $\rightarrow$ $O1 = f(h1*w5 + h2*w6 + b2)$



$\leftarrow$ 反向訓練



求 w_5 時只需考慮來自 E_{o1} 的影響

求 w_1 時要考慮來自 E_{o1} 和 E_{o2} 的影響

影像來源：<https://www.cnblogs.com/charlotte77/p/5629865.html>

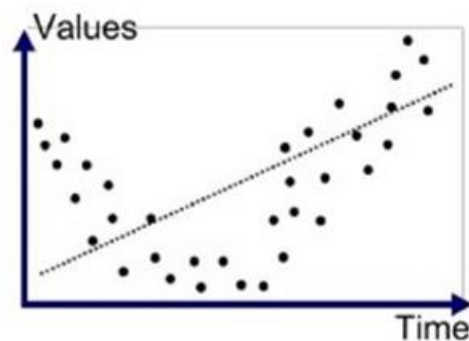
數值擬合

$$y = f(x)$$

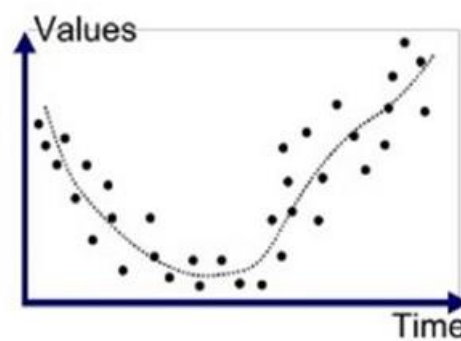
$$f(x) = ax + b \text{ (直線)}$$

$$f(x) = ax^2 + bx + c \text{ (二次曲線)}$$

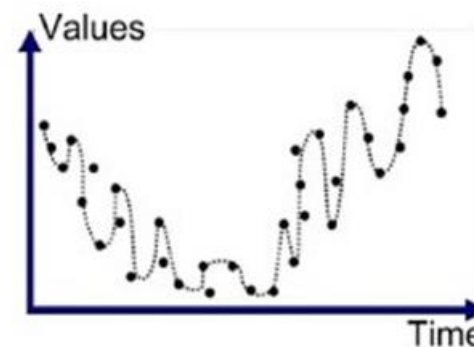
$$f(x) = ax^n + bx^{n-1} + \dots + c \text{ (n次曲線)}$$



Underfitted
未擬合



Good Fit/Robust
良好擬合



Overfitted
過擬合

輸出格式

➤ 獨熱碼(One Hot Encoding)

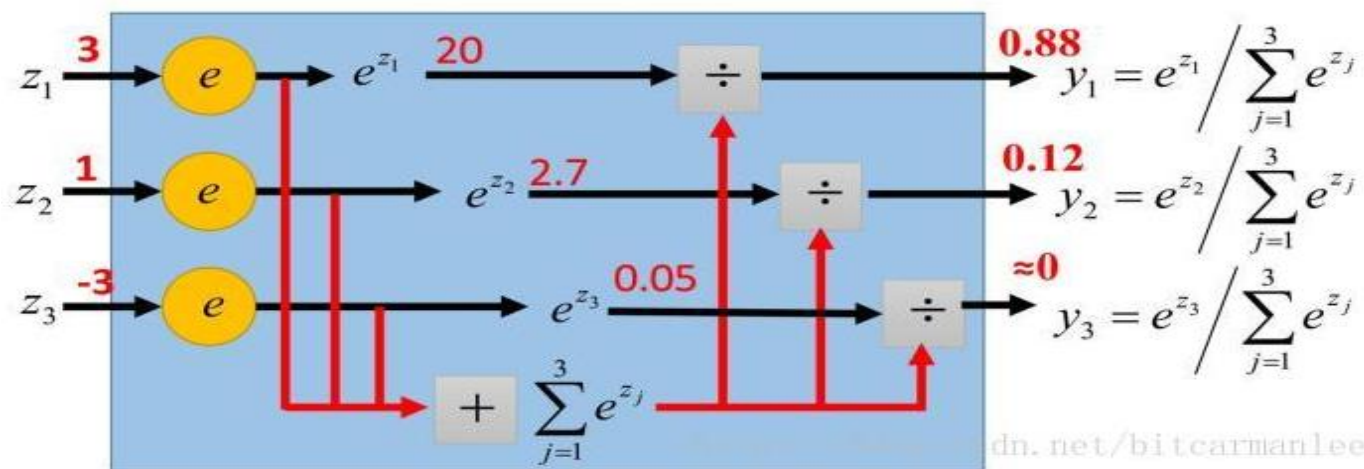
- 每個分類最終結果只有0和1，適合分類具唯一的屬性。
- 假設有三分類則輸出為 $\{0, 0, 1\}$ 、 $\{0, 1, 0\}$ 、 $\{1, 0, 0\}$

➤ Softmax

- 歸一化指數函數
- 輸出值在0.0~1.0間
- 所有輸出加總為1.0

$$S_i = \frac{e^i}{\sum_j e^j}$$

以三分類為例



損失函數

損失函數 = 實際值和預測值的殘差，亦可視為

實際值 = 預測值 + 誤差值，期望 *loss* 越小越好

$$loss = y - \hat{y} \Rightarrow y = \hat{y} + \epsilon$$

- 常見誤差度量方式
 - 平均絕對誤差 **MAE**
 - 均方誤差 **MSE**
 - 均方根誤差 **RMSE**
 - 交叉熵 **Cross-Entropy**

- 進階誤差度量方式
 - Huber Loss
 - Focal Loss
 - Center Loss
 - Triplet Loss

損失函數 — MAE

MAE (Mean-Absolute Error, L1 Loss)

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

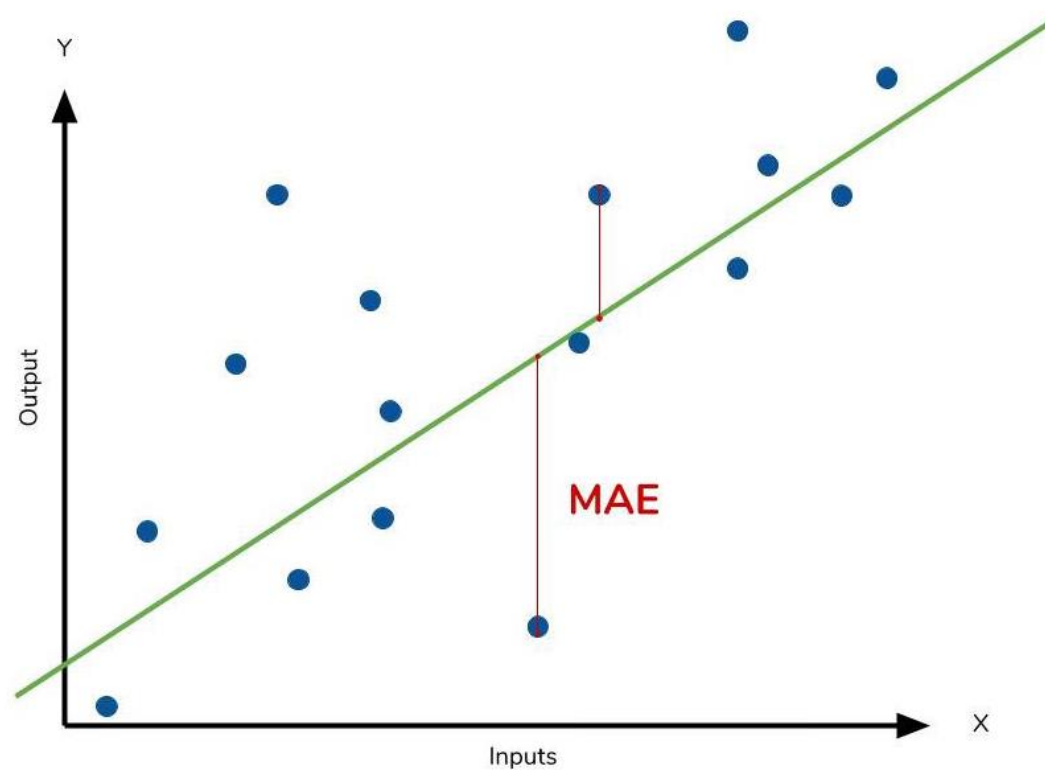
Divide by the total number of data points

Actual output value

Predicted output value

Sum of

The absolute value of the residual



資料來源：<https://www.dataquest.io/blog/understanding-regression-error-metrics/>

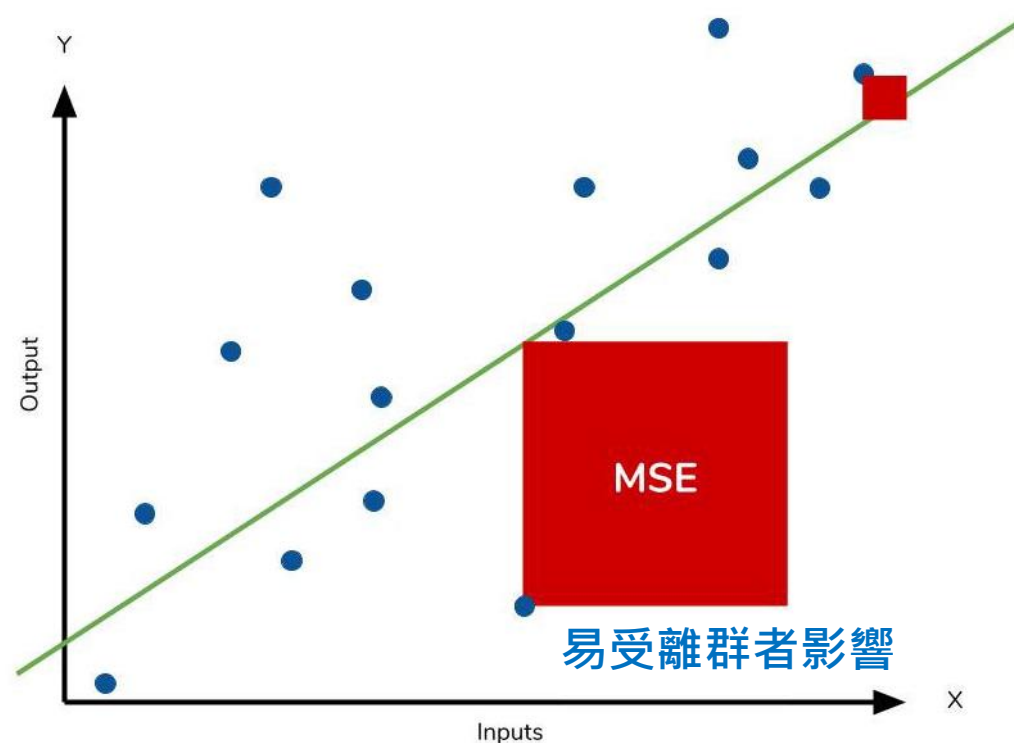
損失函數 — MSE, RMSE

MSE (Mean-Square Error, L2 Loss)

RMSE (Root-Mean-Square Error)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$



資料來源：<https://www.dataquest.io/blog/understanding-regression-error-metrics/>

損失函數 — Cross-Entropy

➤ 交叉熵(cross-entropy)

熵，即亂度，資訊越混亂時熵值越高。

分類問題，交叉熵值越小越有序，即越接近目標。

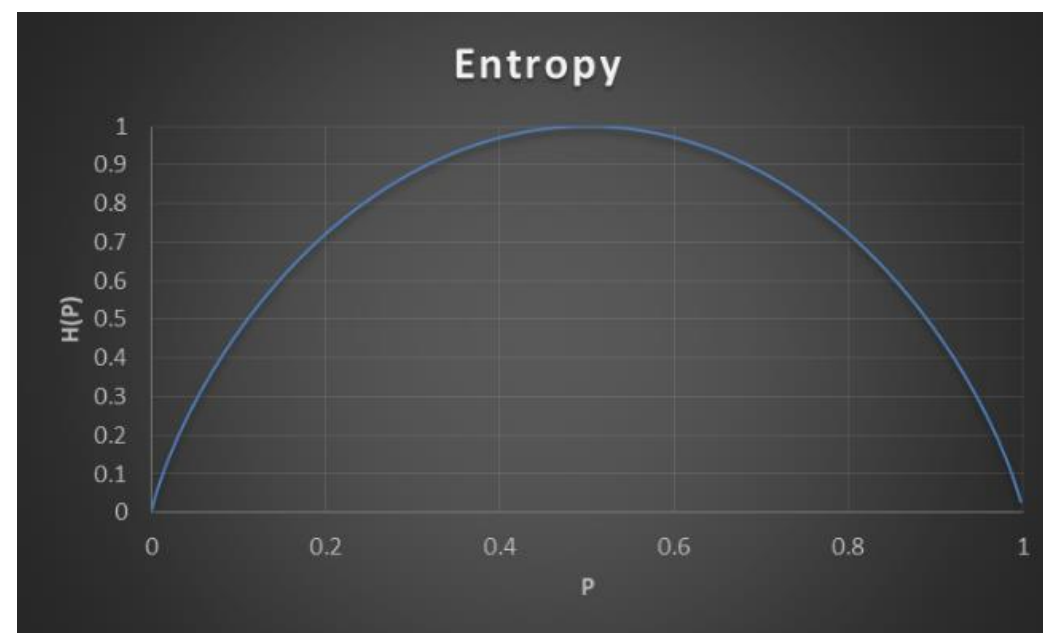
熵

$$H(X) = \sum_i -p_i \log_2(p_i)$$

交叉熵

$$H = \sum_{c=1}^C \sum_{i=1}^n -y_{c,i} \log_2(p_{c,i})$$

類別 資料數



(影像來源)

梯度下降 — 極值、梯度

- 微分找極值，一階微分為零有極值。
- 二階微分大於零有極小值，反之為極大值。

$$f(x) \quad f'(x) = \frac{\partial f(x)}{\partial x} \quad f''(x) = \frac{\partial f'(x)}{\partial x}$$

- 多維向量找極值即對所有元素進行偏微分，求得「**梯度**」
- $$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}, \nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_d} \end{bmatrix}$$

資料來源：<https://chih-sheng-huang821.medium.com/>

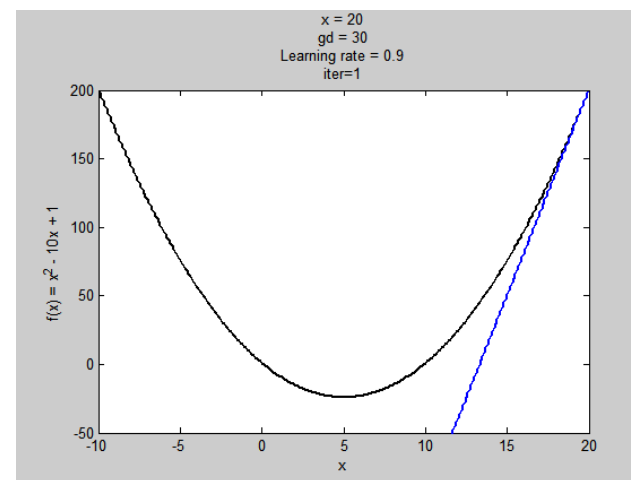
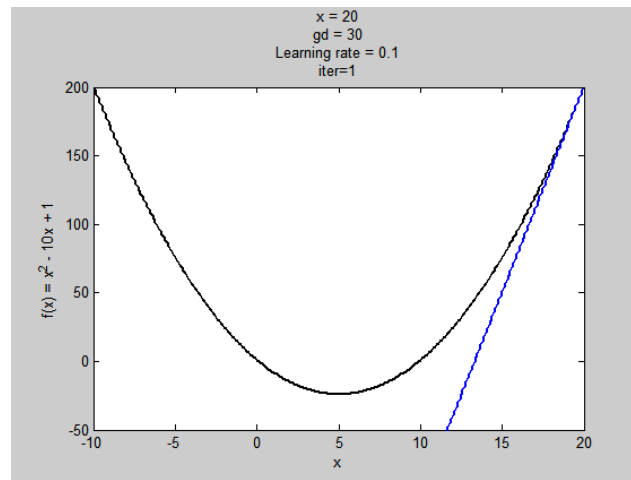
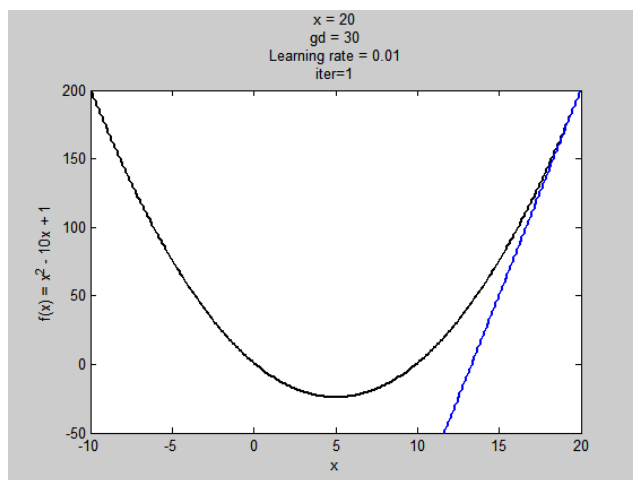
梯度下降 — 學習率

➤ 梯度下降(Gradient Descent)

是一種不斷向梯度方向更新參數找解的方法

其中 t 為第幾次更新， r 為學習率。

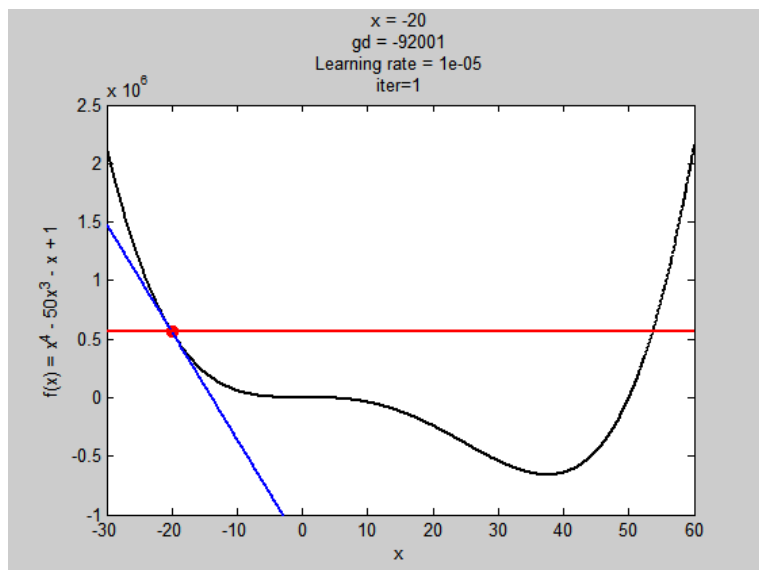
$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \gamma \nabla f(\mathbf{x}^{(t)})$$



資料來源：<https://chih-sheng-huang821.medium.com/>

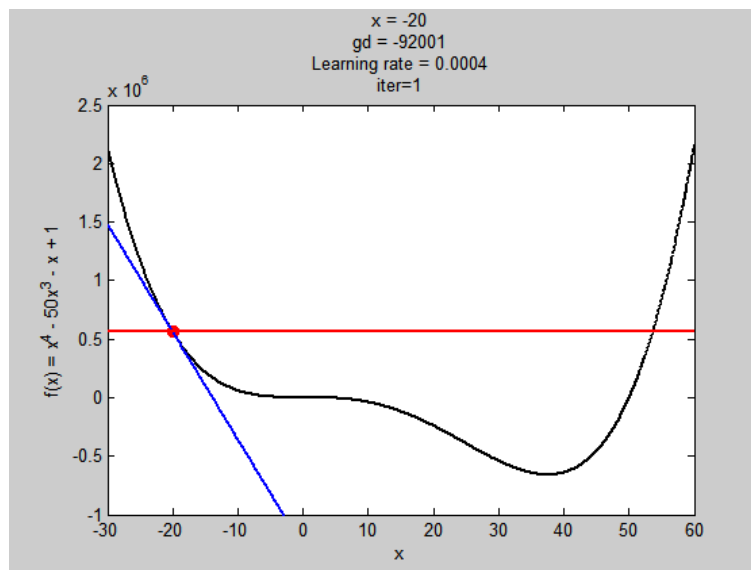
梯度下降 — 固定學習率

學習率過小



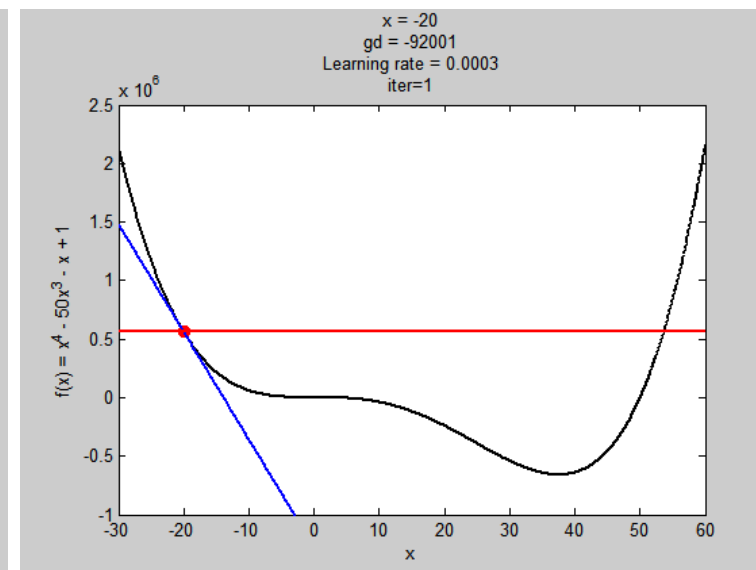
易陷入區域極小值

學習率過大



易產生振盪

學習率適中



快速收斂到最小值

資料來源：<https://chih-sheng-huang821.medium.com/>

梯度下降 — 常見優化器調整方式

$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}$$

學習率(α) 梯度(∇)

Optimiser	Year	Learning Rate	Gradient
Momentum	1964		✓
AdaGrad	2011	✓	
RMSprop	2012	✓	
Adadelta	2012	✓	
Nesterov	2013		✓
Adam	2014	✓	✓
AdaMax	2015	✓	✓
Nadam	2015	✓	✓
AMSGrad	2018	✓	✓

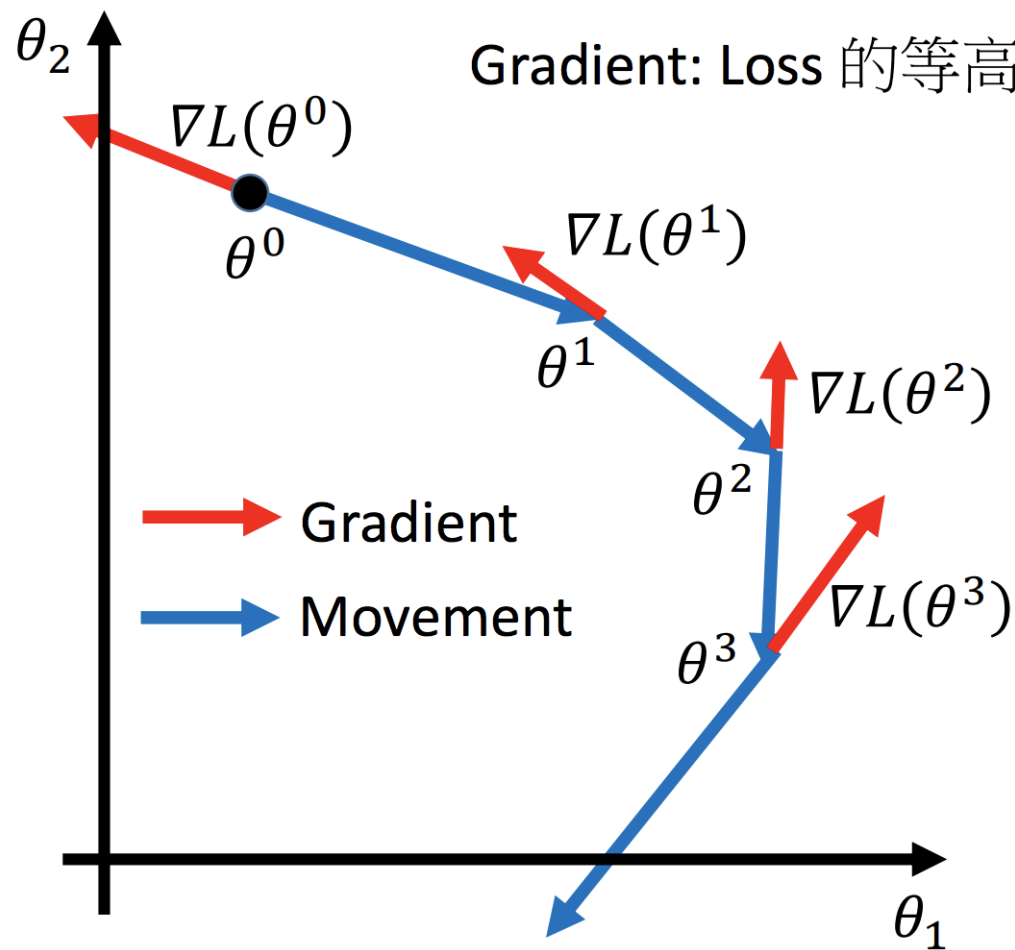
資料來源：<https://towardsdatascience.com/10-gradient-descent-optimisation-algorithms-86989510b5e9>

梯度下降 — BGD / SGD / MBGD

- **批量梯度下降法(Batch Gradient Descent)**
 - 全部樣本計算平均損失函數後再求梯度進行下降更新。
- **隨機梯度下降法(Stochastic Gradient Descent)**
 - 每個樣本求得損失函數就進行更新，由於樣本是隨機的，所以就稱為隨機梯度下降法。
- **小批量梯度下降法(Mini-Batch Gradient Descent)**
 - 搭配指定數量樣本(Batch Size)計算平均損失函數後再求梯度進行下降更新。
- **缺點：**很難找到一個適合的固定學習率或批量大小來滿足各種情況。

資料來源：<https://chih-sheng-huang821.medium.com/>

梯度下降 — 梯度更新



資料來源：<https://medium.com/chung-yi>

梯度下降 — Adagrad

- 自適應梯度（自動學習率調整）法：Adagrad
 - 隨時間令學習率逐漸自動變小，以減少振盪，快速收斂。

$$\eta^t = \frac{\eta}{\sqrt{t+1}} \quad g^t = \frac{\partial L(\theta^t)}{\partial w}$$

Adagrad

$$w^{t+1} \leftarrow w^t - \frac{\eta^t}{\sigma^t} g^t$$

σ^t : *root mean square* of the previous derivatives of parameter w

Parameter dependent

資料來源：<https://medium.com/chung-yi>

梯度下降 — RMSProp

➤ 均方根傳播法(Root-Mean-Square Propagation)

- 如同Ardgrad除了考量時間因素令學習率逐漸衰減外，另外RMSProp再加入衰減因子 α 。

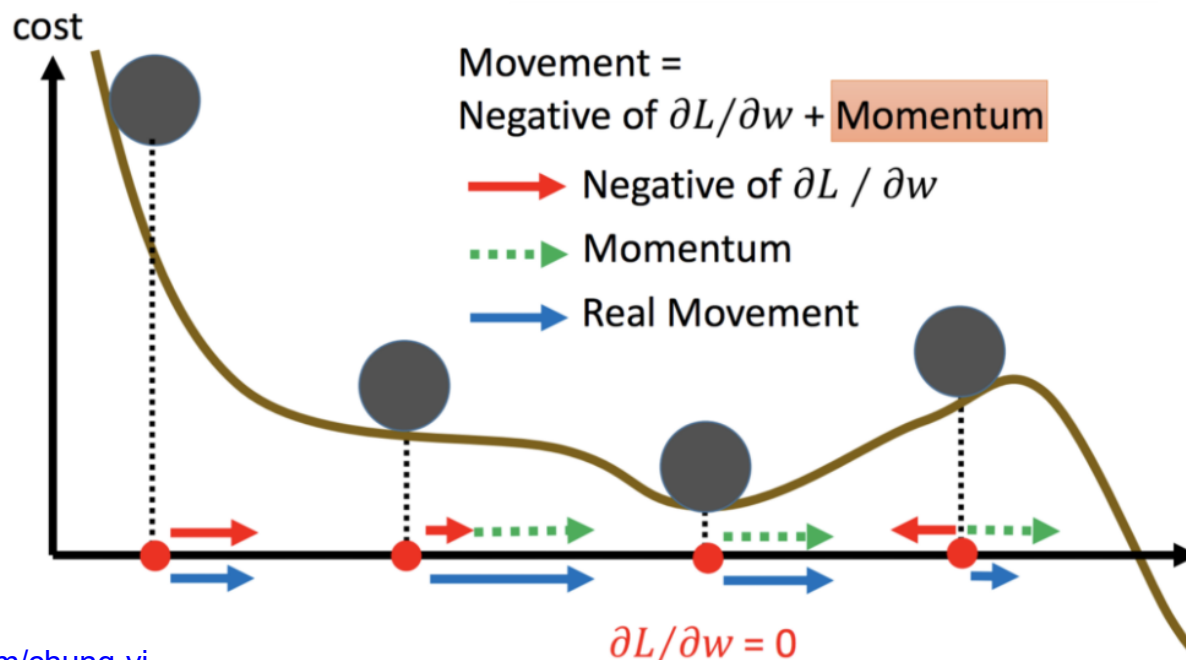
$$\begin{aligned}
 w^1 &\leftarrow w^0 - \frac{\eta}{\sigma^0} g^0 & \sigma^0 &= g^0 \\
 w^2 &\leftarrow w^1 - \frac{\eta}{\sigma^1} g^1 & \sigma^1 &= \sqrt{\alpha(\sigma^0)^2 + (1 - \alpha)(g^1)^2} \\
 w^3 &\leftarrow w^2 - \frac{\eta}{\sigma^2} g^2 & \sigma^2 &= \sqrt{\alpha(\sigma^1)^2 + (1 - \alpha)(g^2)^2} \\
 &\vdots & & \\
 w^{t+1} &\leftarrow w^t - \frac{\eta}{\sigma^t} g^t & \sigma^t &= \sqrt{\alpha(\sigma^{t-1})^2 + (1 - \alpha)(g^t)^2}
 \end{aligned}$$

資料來源：<https://medium.com/chung-yi>

慣性動量

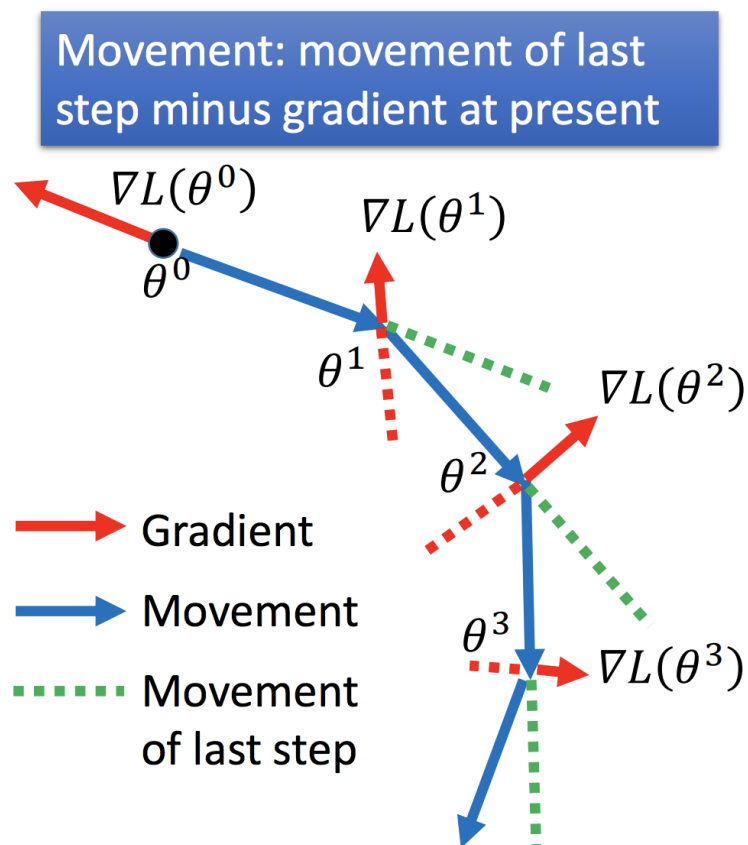
➤ 動量(Momentum)

- 單純梯度下降容易陷入區域最小值。
- 利用物理慣性概念來修正運動方向，即**梯度**和**動量**組成的新向量作為最後移動方向及大小。



資料來源：<https://medium.com/chung-yi>

慣性動量 — Momentum



Start at point θ^0

Movement $v^0=0$

Compute gradient at θ^0

Movement $v^1 = \lambda v^0 - \eta \nabla L(\theta^0)$

Move to $\theta^1 = \theta^0 + v^1$

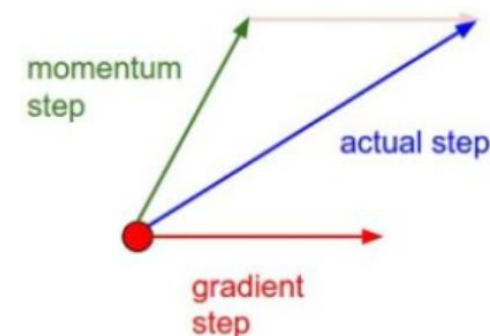
Compute gradient at θ^1

Movement $v^2 = \lambda v^1 - \eta \nabla L(\theta^1)$

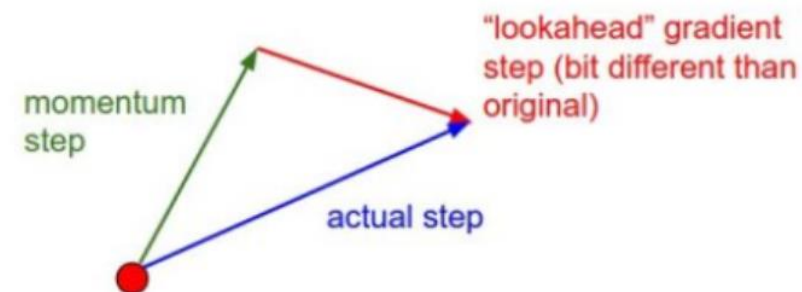
Move to $\theta^2 = \theta^1 + v^2$

Movement not just based on gradient, but previous movement.

Momentum更新



Nesterov Momentum更新



資料來源：<https://medium.com/chung-yi>

慣性動量 — Adam

➤ 自適應動量估測(Adaptive Moment Estimation)

- 相當是加了動量的RMSProp，並在梯度更新時考慮偏差校正(Bias-Correction)
- 結合Adagrad和RMSProp優點，可處理高維及大資料集梯度差異較大的數據。

一階動量

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L_t}{\partial W_t}$$

二階動量

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L_t}{\partial W_t} \right)^2$$

一階校正

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

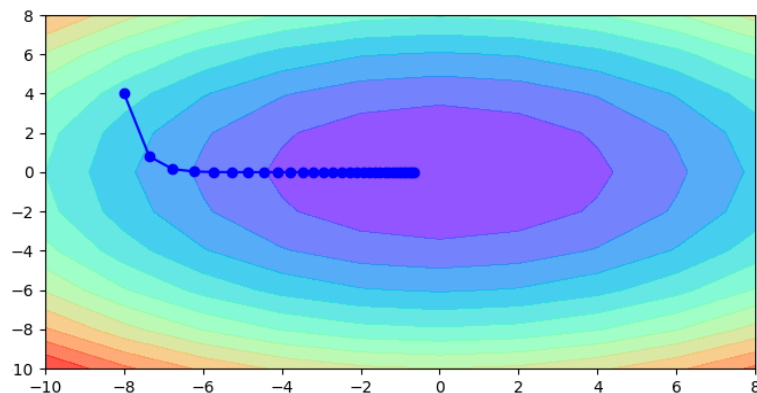
二階校正

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

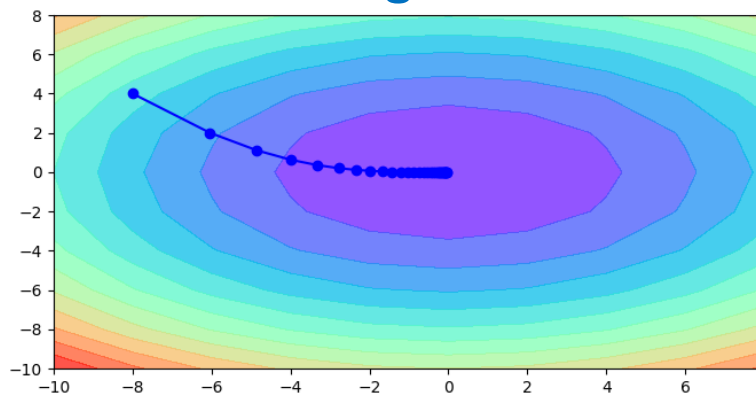
資料來源：<https://medium.com/chung-yi>

慣性動量 — 實驗比較

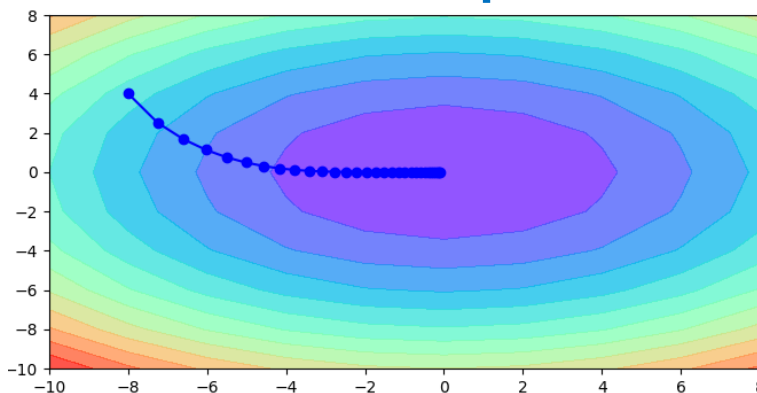
SGD



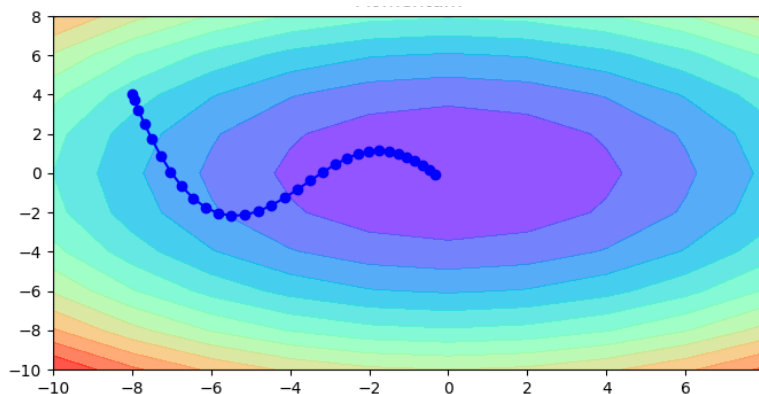
Adgrad



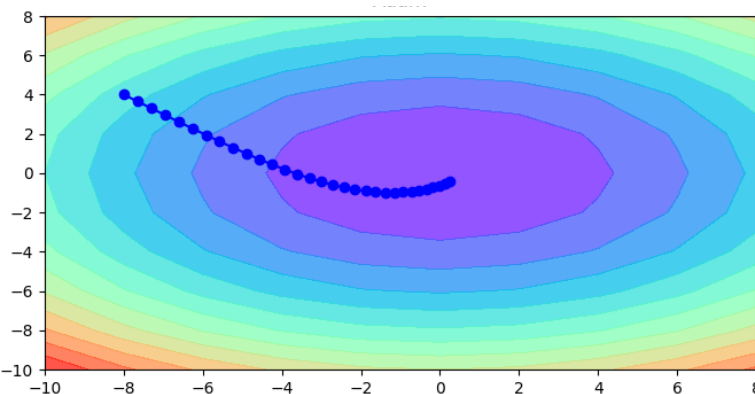
RMSProp



Momentum



Adam

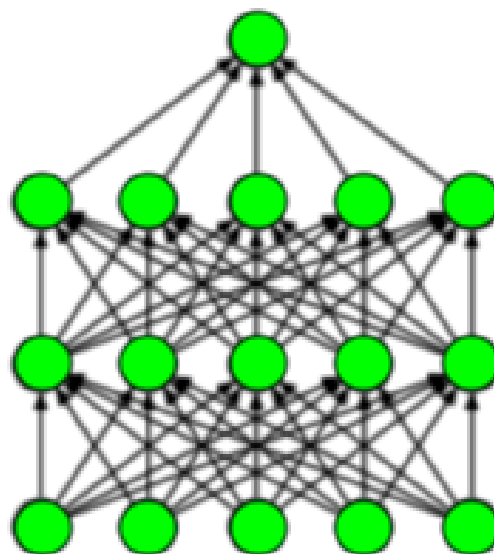


資料來源：<https://medium.com/chung-yi>

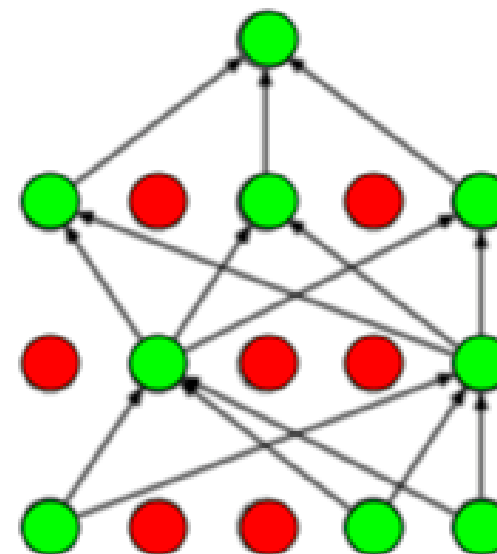
隨機丟棄

➤ Dropout

- Google提出的一種正規化技術，用來防止小資料集**過擬合**。
- 在訓練階段隨機丟棄部份神經元，只更新其它權重。
- 可減少計算數量。



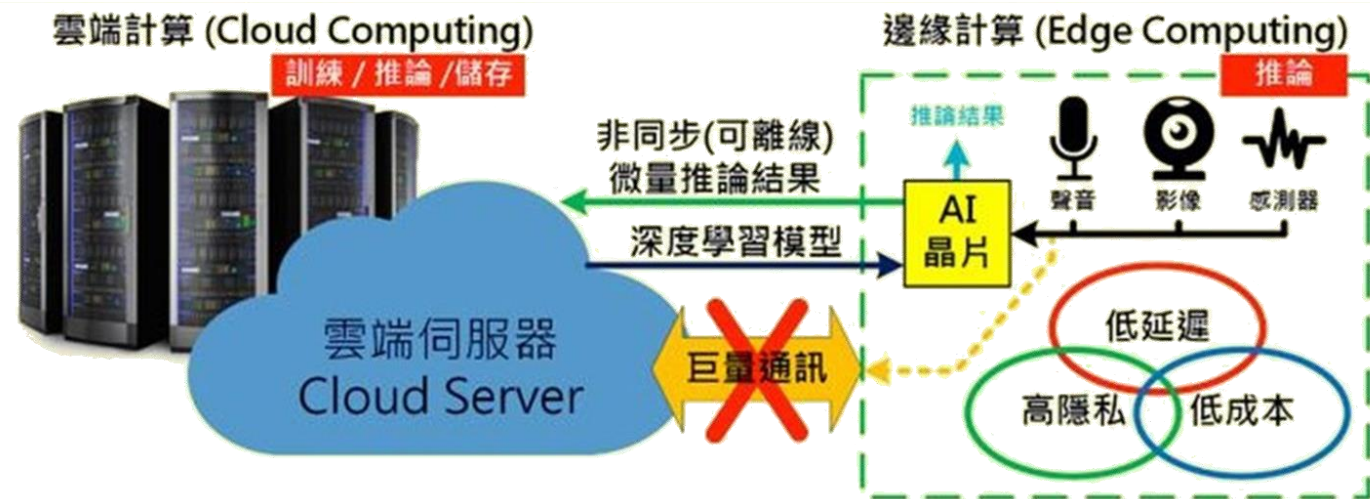
原始網路



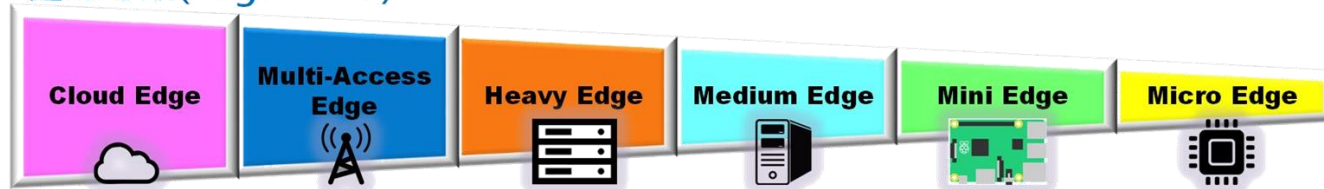
紅點為被隨機丟棄的神經元



Edge AI

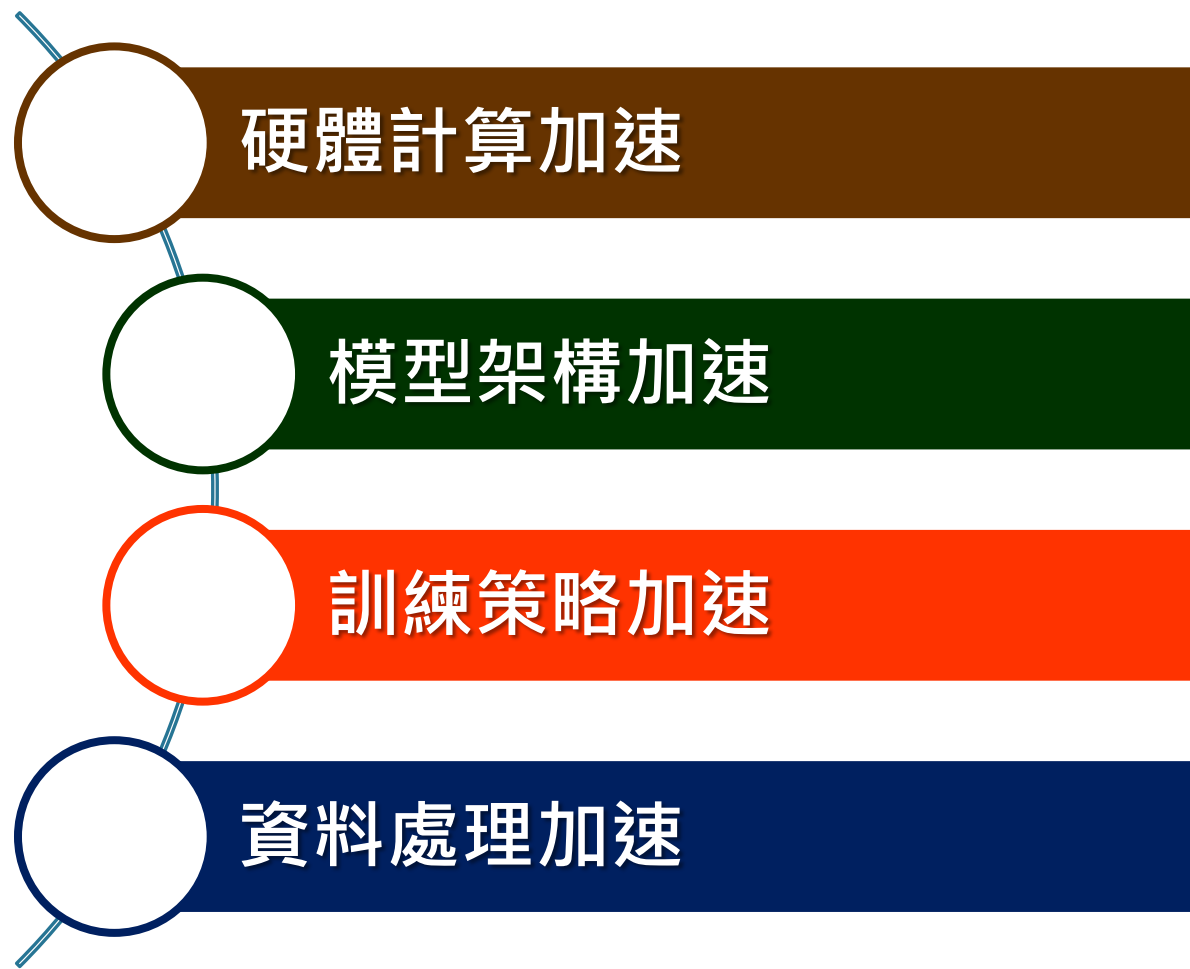
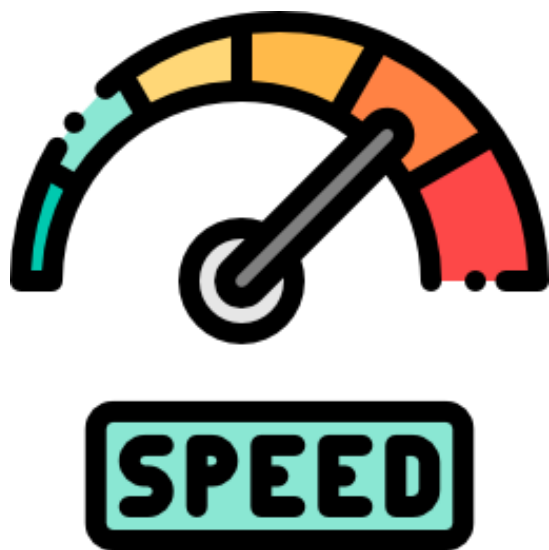
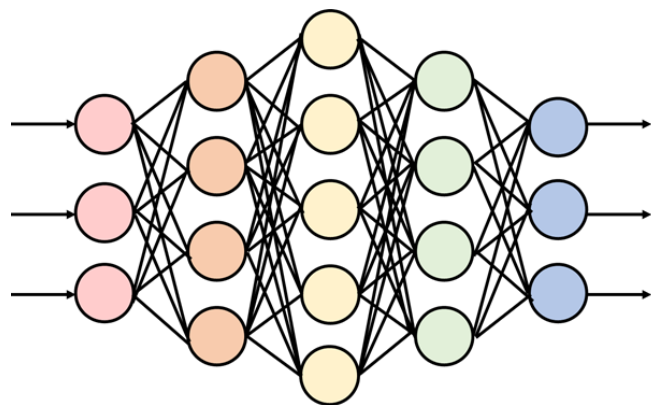


邊緣等級(Edge Level)

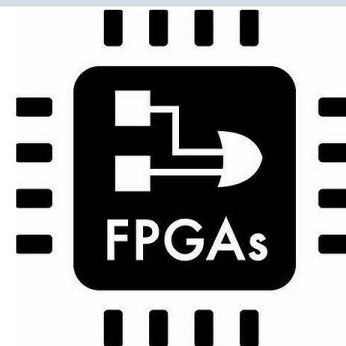
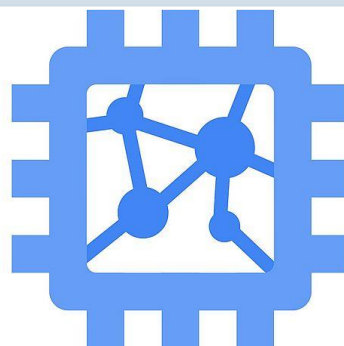
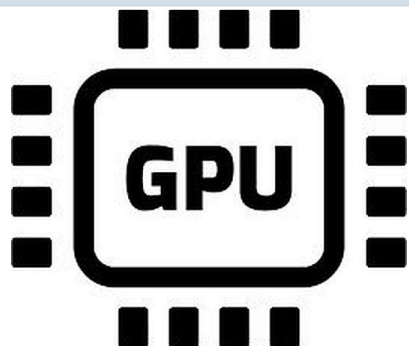


5.2 加速訓練方式

常見模型訓練加速方式



硬體計算加速



Nvidia (CUDA)
AMD (ROCm)
Intel (OpenCL)
Arm (Vulkan)

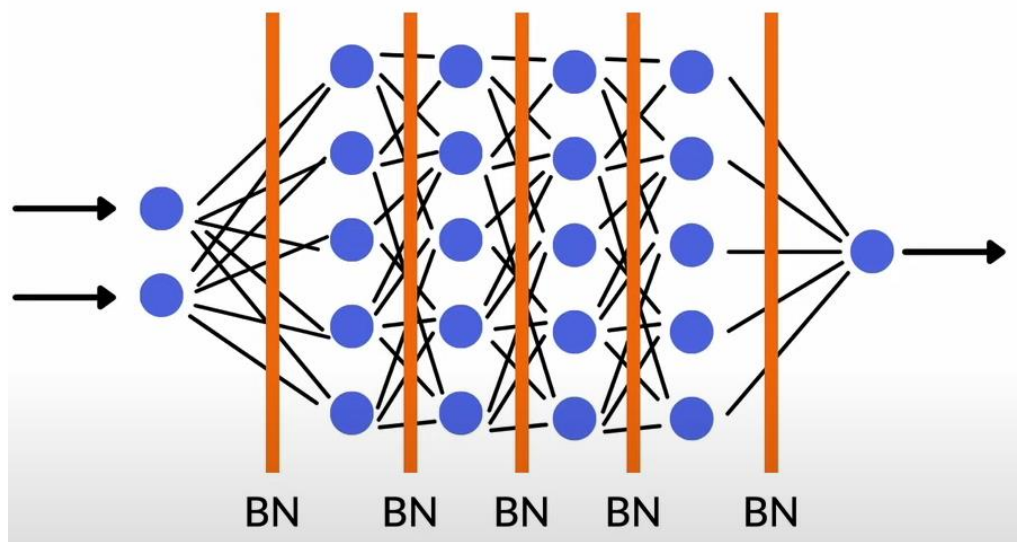
Google

AMD (原Xilinx)
Altera (原Intel)
Lattice
Microchip

模型架構加速 — 批次正規化

➤ 批次正規化 (Batch Normalization, BN)

- 可以在每一層間加入，令資料收斂到 **0.0 ~ 1.0** 間，穩定資料分佈範圍，同時緩解訓練過程的梯度消失及爆炸。
- 允許使用更大的學習率，同時改善權重初始化的敏感度。



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

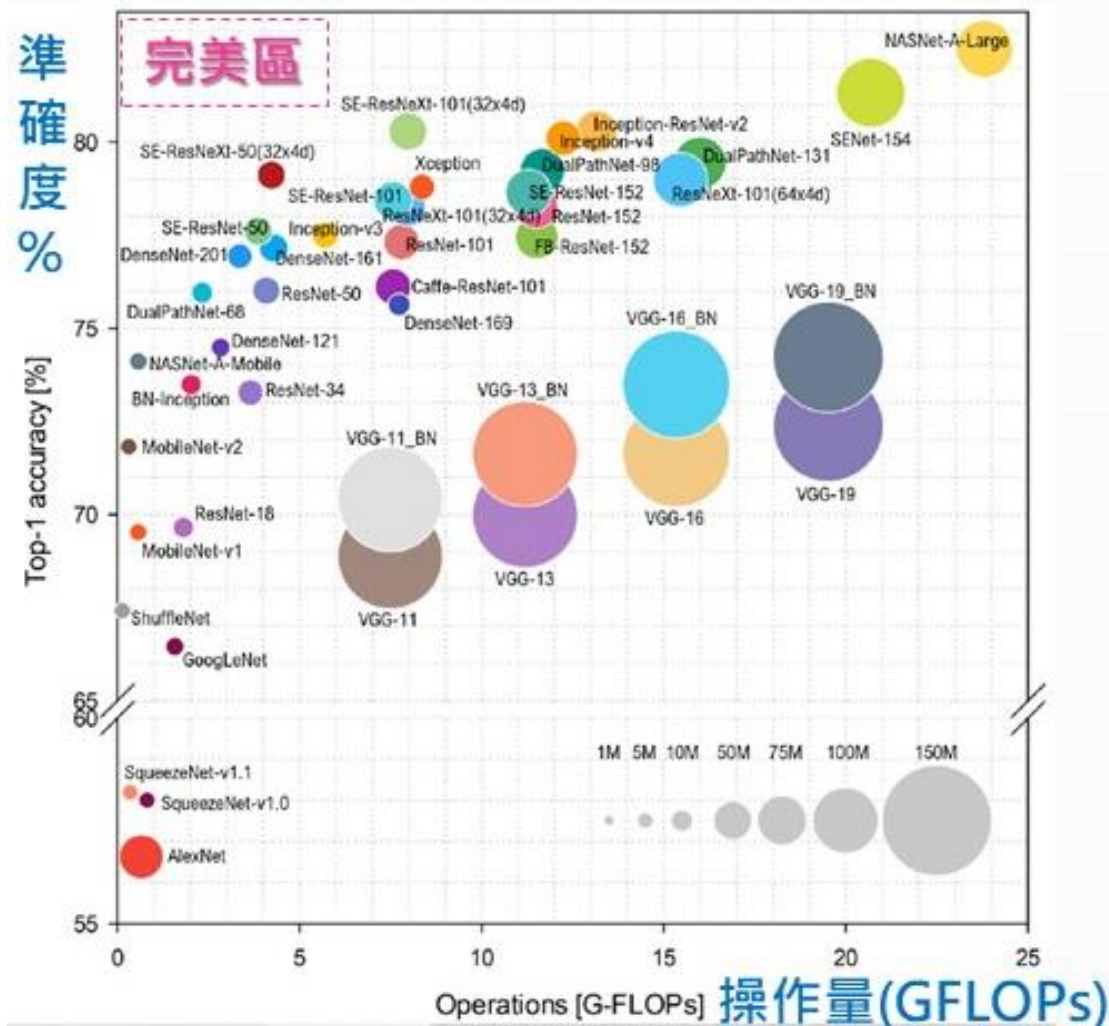
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

資料來源：<https://arxiv.org/abs/1502.03167>

模型架構加速 — 低複雜度模型



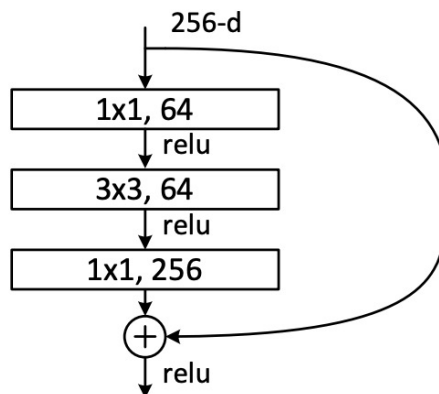
➤ 重要評估指標

- 準確率(Accuracy)
- 記憶體使用量(Memory Footprint)
- 參數量(Parameter)
- 操作數(Operations Count)
- 推理時間(Inference Time)
- 功耗(Power Consumption)

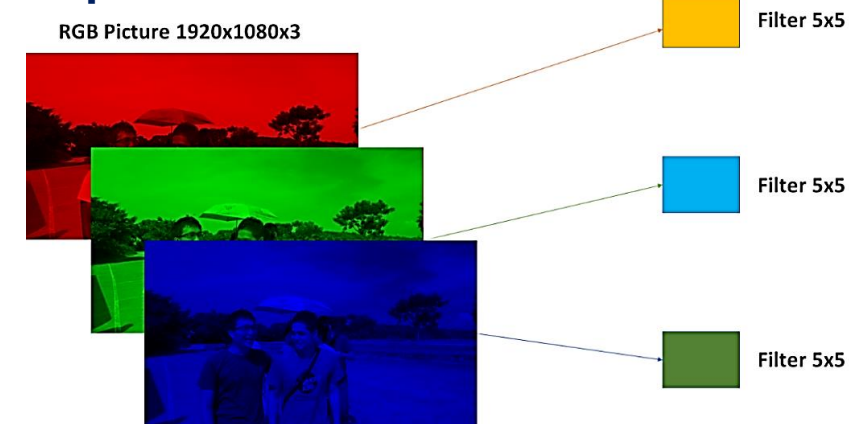
模型小參數小不代表計算量少，計算量少通常推論時間及功耗較小。

模型架構加速 – 輕量化卷積

殘差網路 (**Residual Neural Network**) 如 ResNet, MobileNet, ShuffleNet, EfficientNet 等。

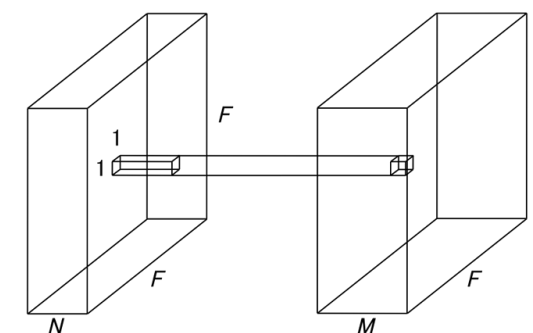


Depthwise Convolution



$$1920 \times 1080 \times 3 \times 5 \times 5 \times 3 = 466,560,000$$

Pointwise Convolution



$$1920 \times 1080 \times N \times M$$

- 輸出入尺寸相同(F)
- 可輕易升降維(M)

1920*1080



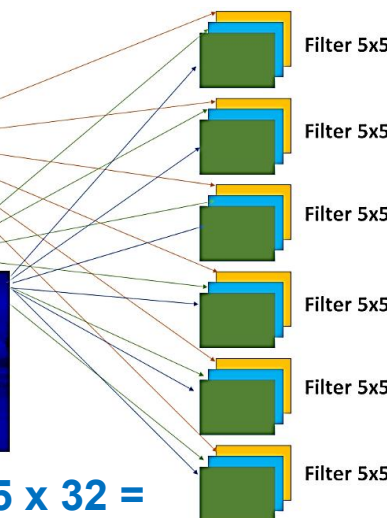
32 個 5*5 Filter

傳統 CNN

RGB Picture 1920x1080x3



$$1920 \times 1080 \times 3 \times 5 \times 5 \times 32 = 4,976,640,000$$



影像來源：<https://chtseng.wordpress.com/2019/04/05/depth-wise-and-point-wise-cnn/>

模型架構加速 — 自動學習

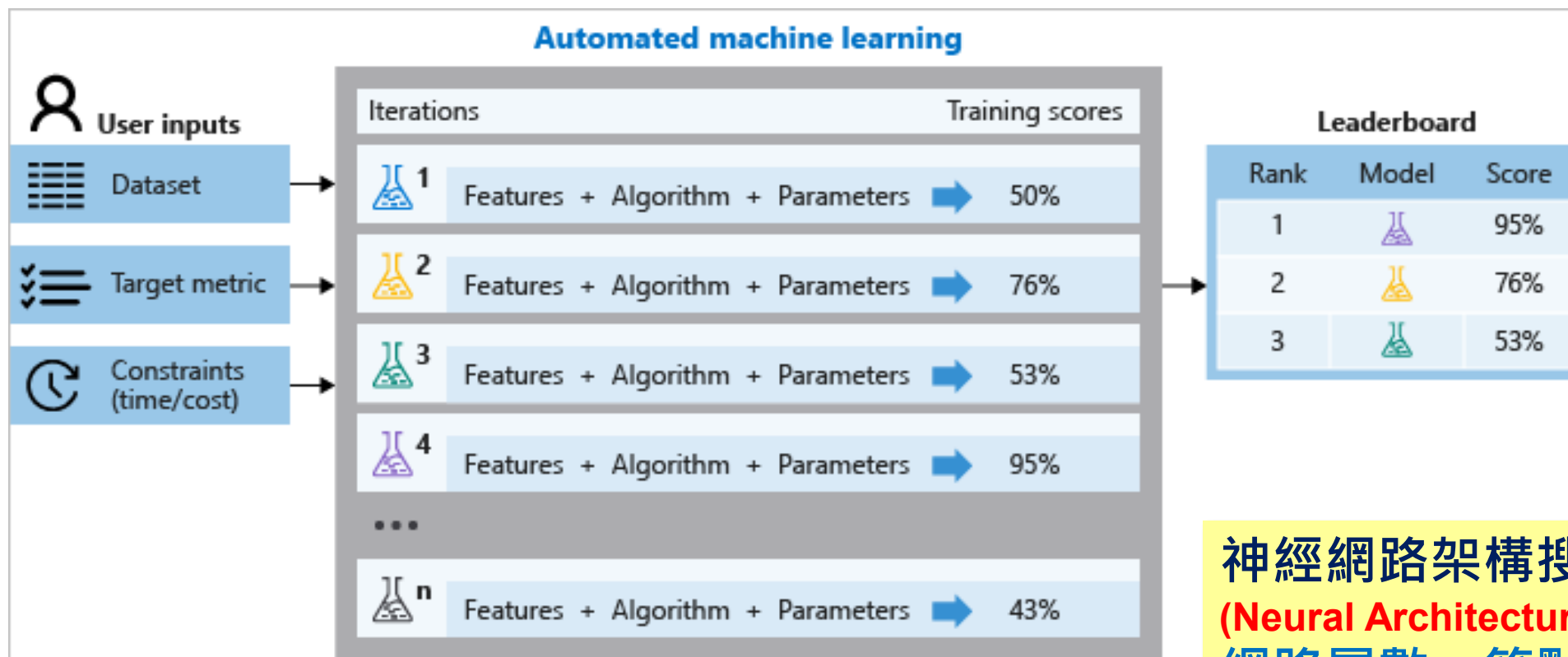
➤ 自動學習 (AutoML)

- 使用者僅需準備已標註資料集
- 不需資料科學家介入、免程式開發、自動找出最合適算法
- 自動反覆調整參數使模型效率達到最高
- 適合迴歸分析、分類及時間預測等算法。

➤ 常見框架及平台

- Google GCP, Amazon AWS, Microsoft Azure 付費式 AutoML
- AutoKeras、AutoSKlearn、Automl-gs
- Auto-Weka、FeatureTools、Ludwig
- MLBox、TPOT、H2O AutoML
- Neural Network Intelligence、TransmograpAI
- **Edge Impulse EON**

自動學習 — 基本框架



主要工作：資料預處理、特徵工程、特徵提取、特徵選擇、模型訓練、算法選擇、超參數優化

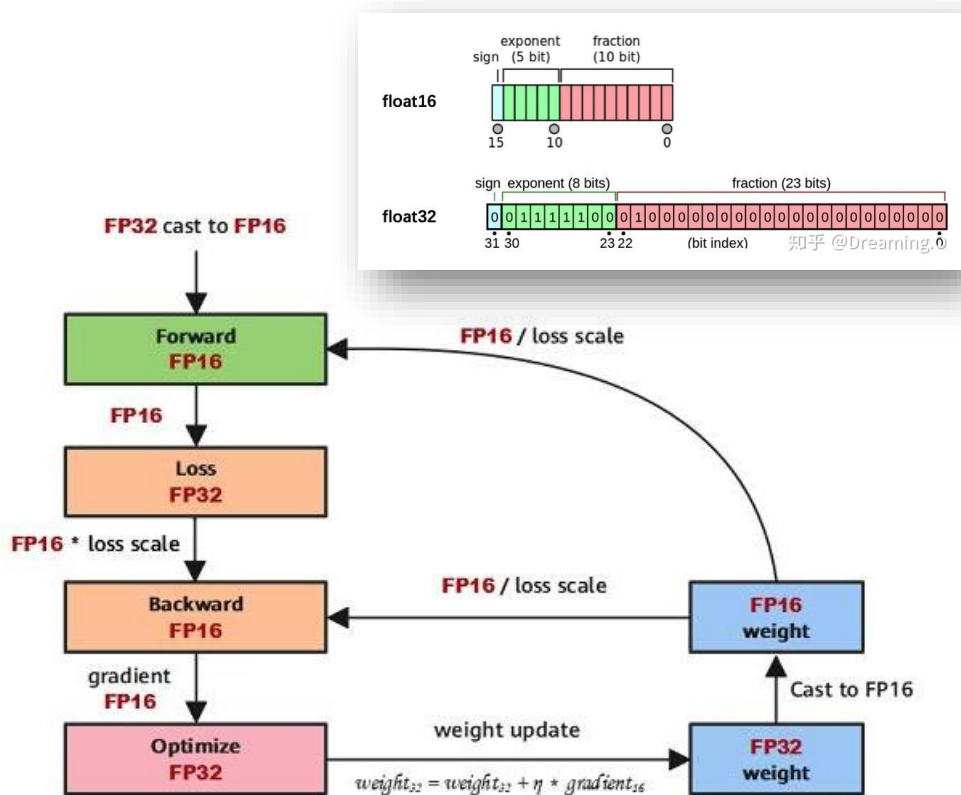
神經網路架構搜尋
(Neural Architecture Search, NAS)
網路層數、節點數、連結方式、優化器、效能評估、硬體限制等

資料來源：<https://docs.microsoft.com/zh-tw/azure/machine-learning/concept-automated-ml>

訓練策略加速 – 混合精度 / 優化器

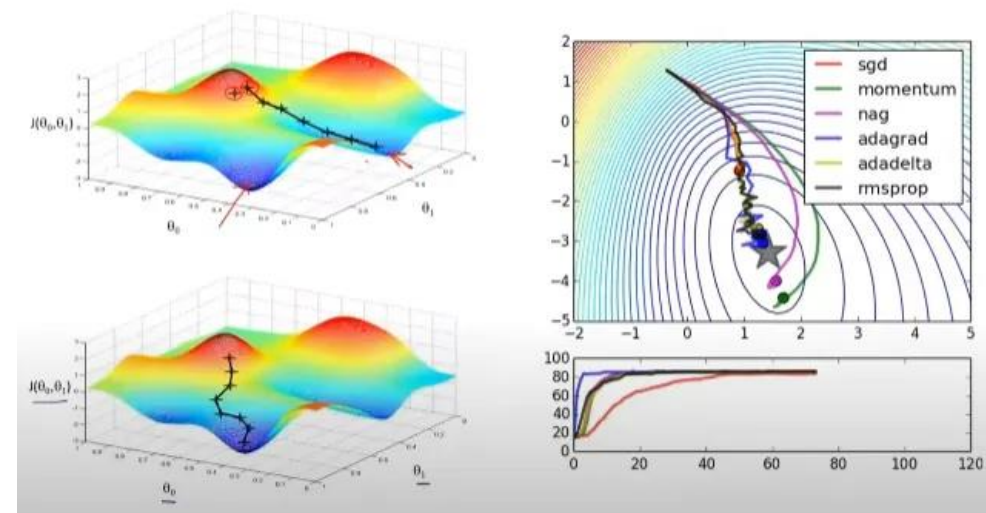
➤ 混合精度

可提高2~3倍訓練速度



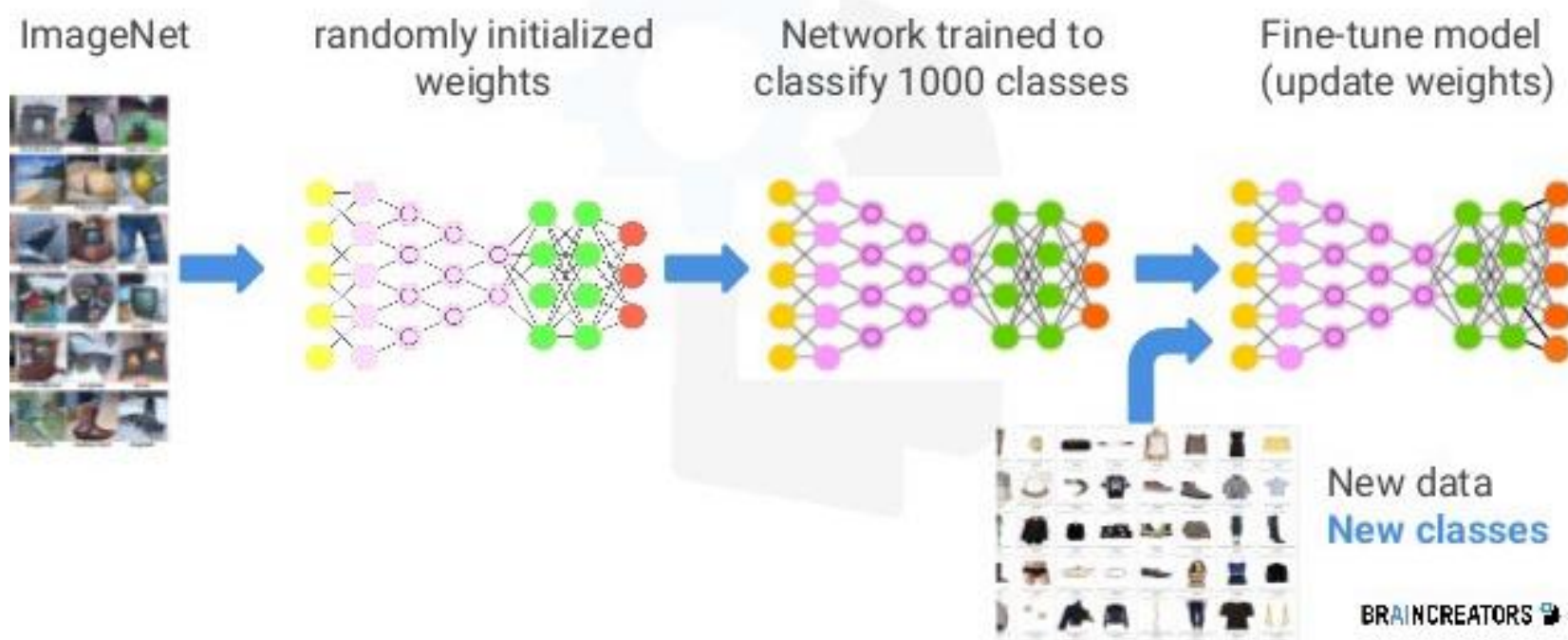
➤ 優化器選用

- 梯度下降型：BGD, SGD, MBGD ...
- 自適應型：Adagrad, RMSprop, Adam ...



訓練策略加速 — 遷移學習

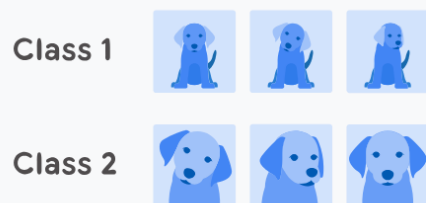
- **遷移式學習(Transfer Learning)**
 - 利用已訓練好的模型，拆解最後幾層，換上新的再重訓。
 - 適用資料集不足或想加快訓練時間場合。



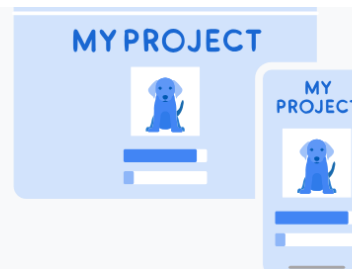
資料來源：<https://madhuramiah.medium.com/deep-learning-using-resnets-for-transfer-learning-d7f4799fa863>

遷移式學習 — 應用程式

➤ Google Teachable Machine



TRAIN MODEL



1 Gather

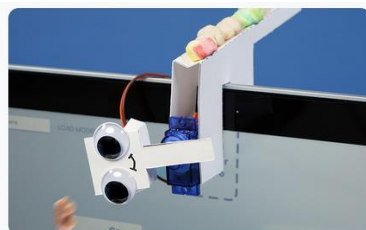
Gather and group your examples into classes, or categories, that you want the computer to learn.

2 Train

Train your model, then instantly test it out to see whether it can correctly classify new examples.

3 Export

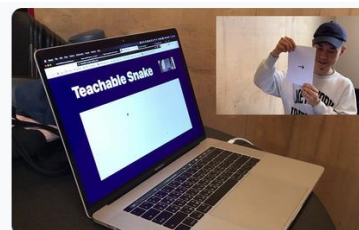
Export your model for your projects: sites, apps, and more. You can download your model or host it online.



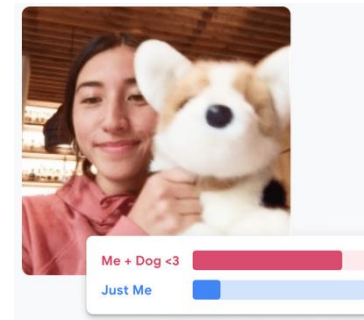
Tiny Sorter



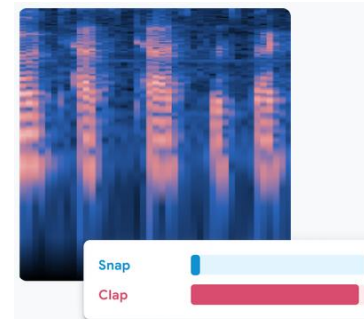
Project Euphonia



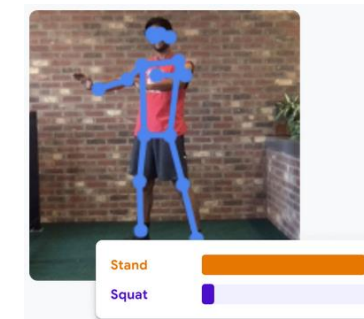
Teachable Snake



影像



聲音

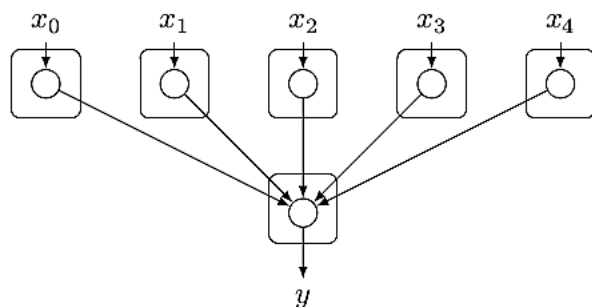


姿態

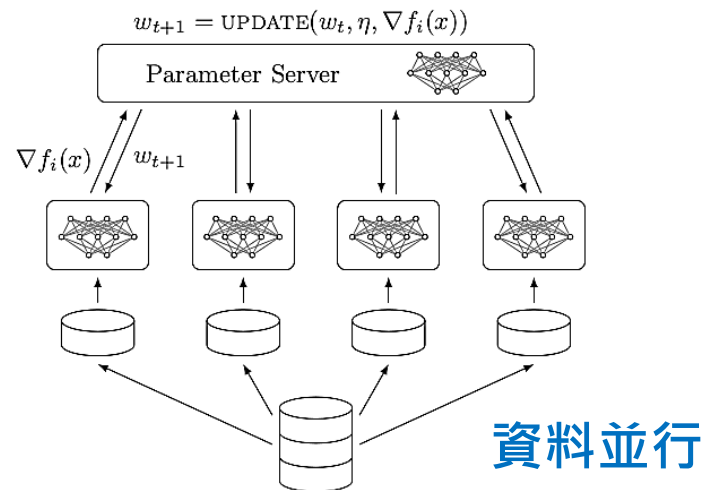
<https://teachablemachine.withgoogle.com/>

訓練策略加速 – 分散式學習

- 分散式訓練/學習(Distributed Training / Learning)
 - 模型並行(Model Parallelism)
 - 資料並行(Data Parallelism)
 - 引數平均(Model Averaging)
 - 非同步隨機梯度下降(Asynchronous SGD)



模型並行

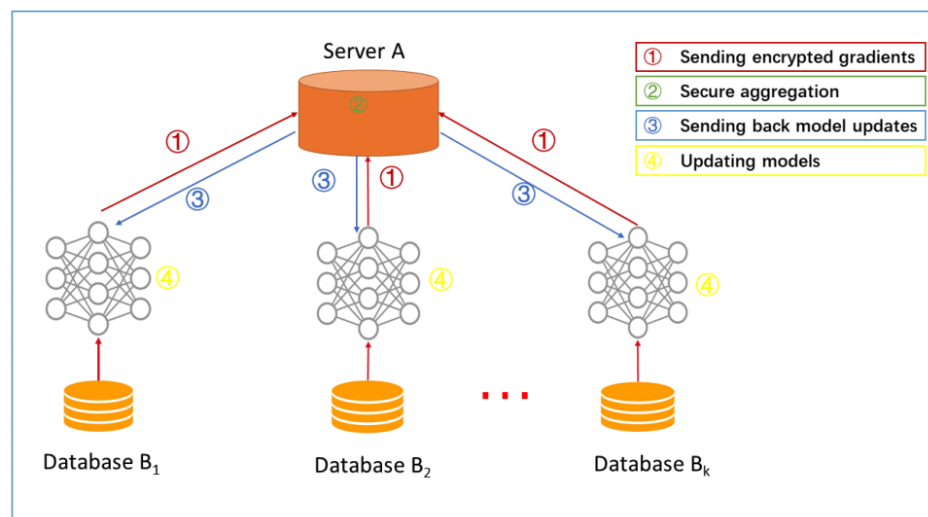


資料並行

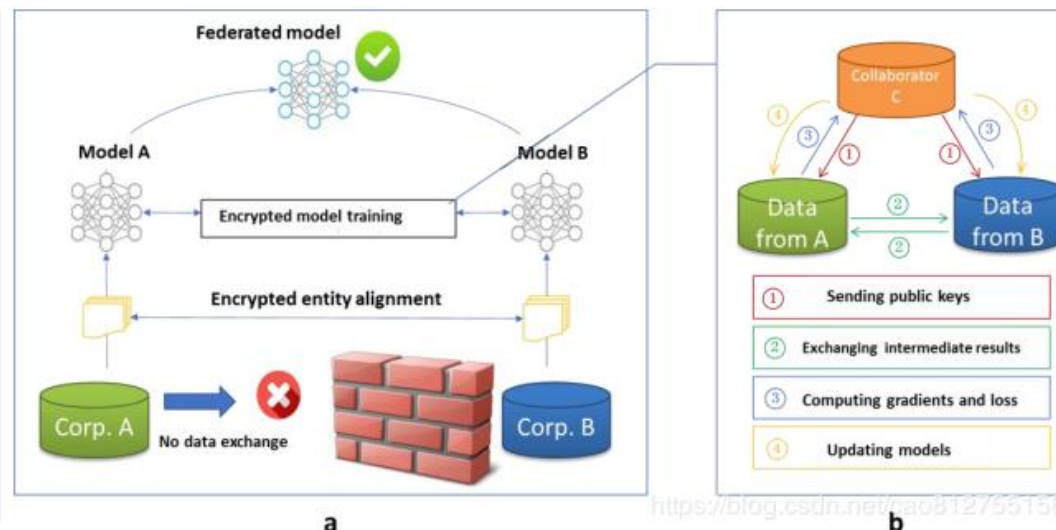
資料來源：<https://joerihermans.com/ramblings/distributed-deep-learning-part-1-an-introduction/>

訓練策略加速 — 聯邦學習

- **聯合/聯邦/聯盟學習(Federated Learning)**
 - 資料共享訓練但不提供原始資料，具高隱私性。
 - 模型各自訓練但共享訓練成果，再重新佈署調整參數。



水平式



垂直式

資料來源：<https://medium.com/disassembly/architecture-of-federated-learning-a36905c1d225>

資料處理加速 — 資料管道 / 資料降維

➤ 資料管線優化

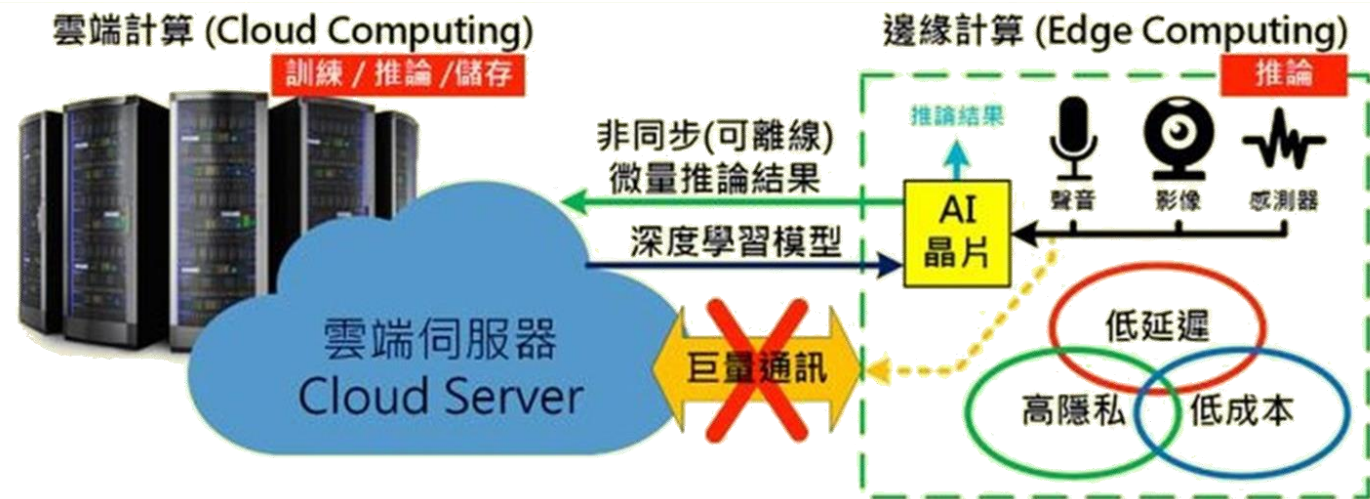
- 預取 (Prefetching) , GPU 在計算時讓 CPU 提前準備資料。
- 並行載入與轉換, 利用多執行緒同時載入資料並轉換 (如影像解壓縮)
- 改變資料格式 (如 TFRecord), 減少磁碟搜尋時間, 提高讀取效率。

➤ 資料降維

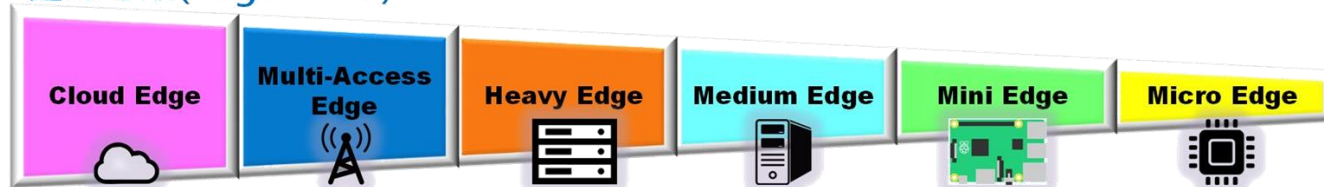
- 改變資料精度, 減少儲存及計算空間, 如FP32降至FP16, BF16。
- 特徵提取, 如時域轉頻域。
- 影像尺寸縮小或品質改善。
- 將資料增強 (Argumentation) 減少資料集不平衡。
- 縮小資料集使其更具代表性資料並提高批次取樣量。



Edge AI



邊緣等級(Edge Level)



5.3 模型推論優化

資料表示位數減少

優點：儲存空間減少

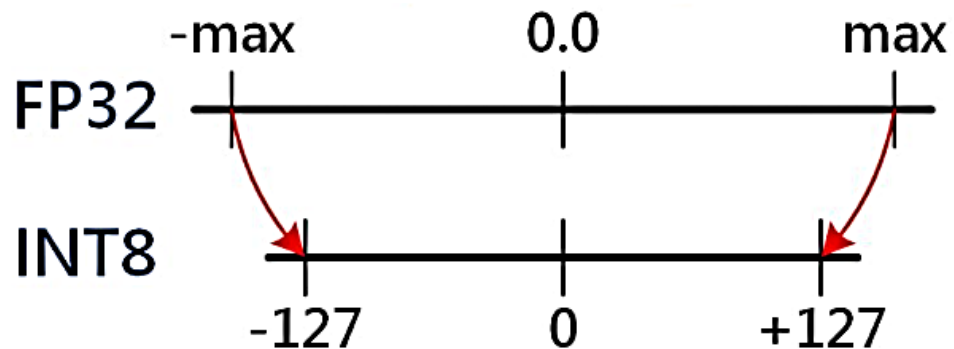
缺點：動態範圍變小



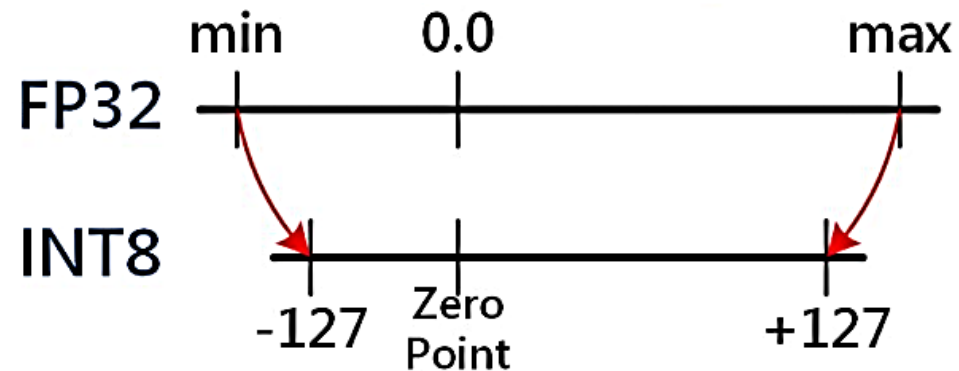
數值量化－量化方式

線性量化

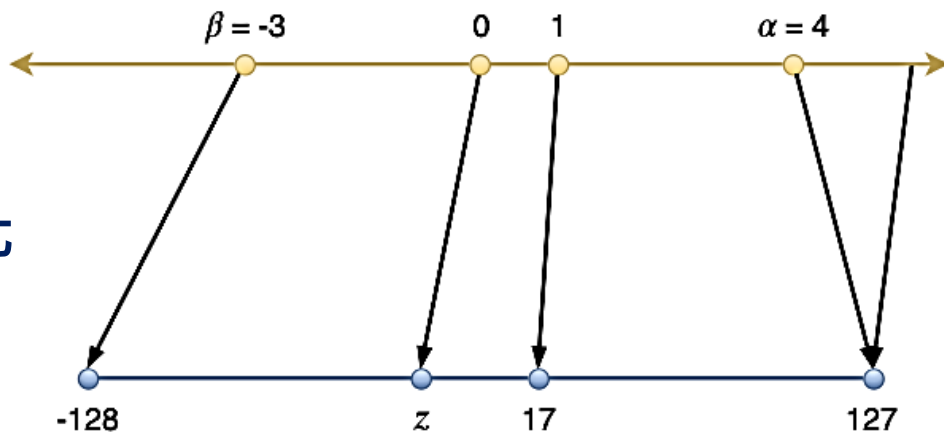
對稱量化(Symmetric Quantization)



非對稱量化(Asymmetric Quantization)

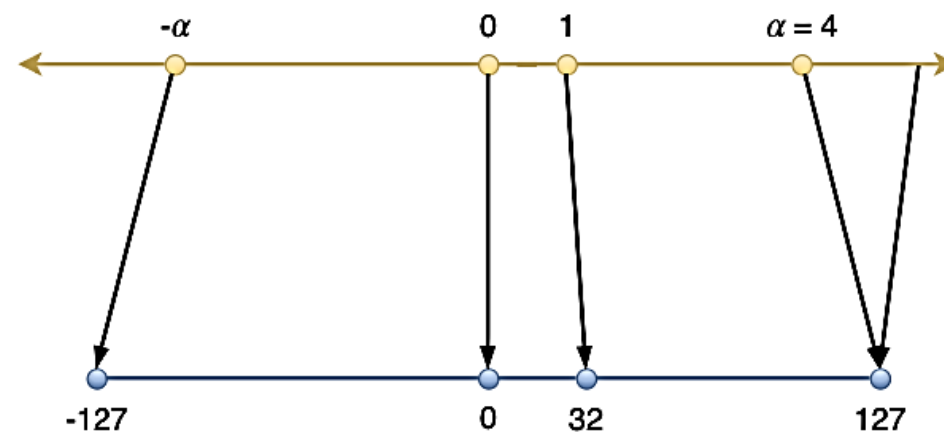


仿射量化(Affine Quantization)



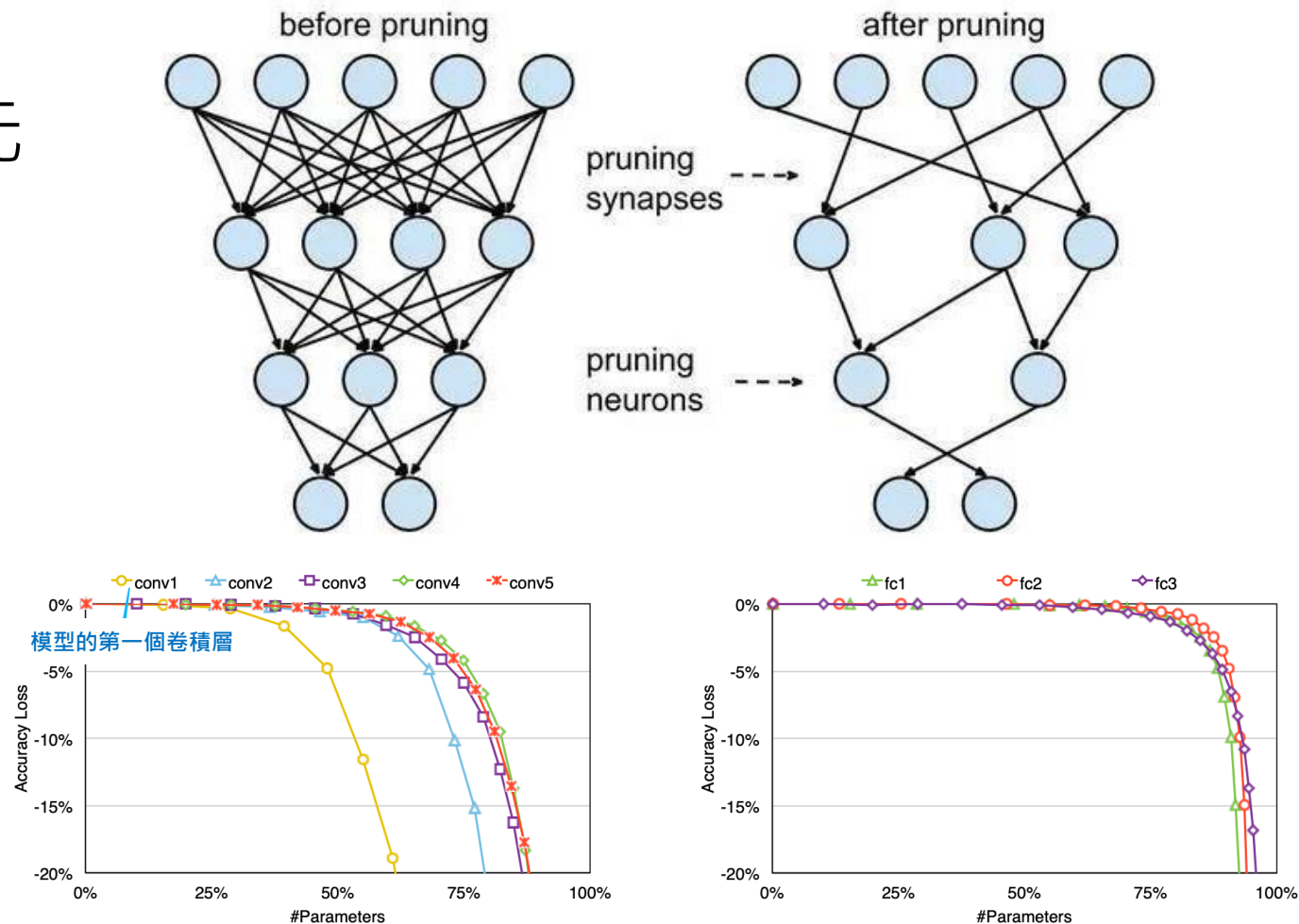
非線性量化

尺度量化(Scale Quantization)



模型剪枝

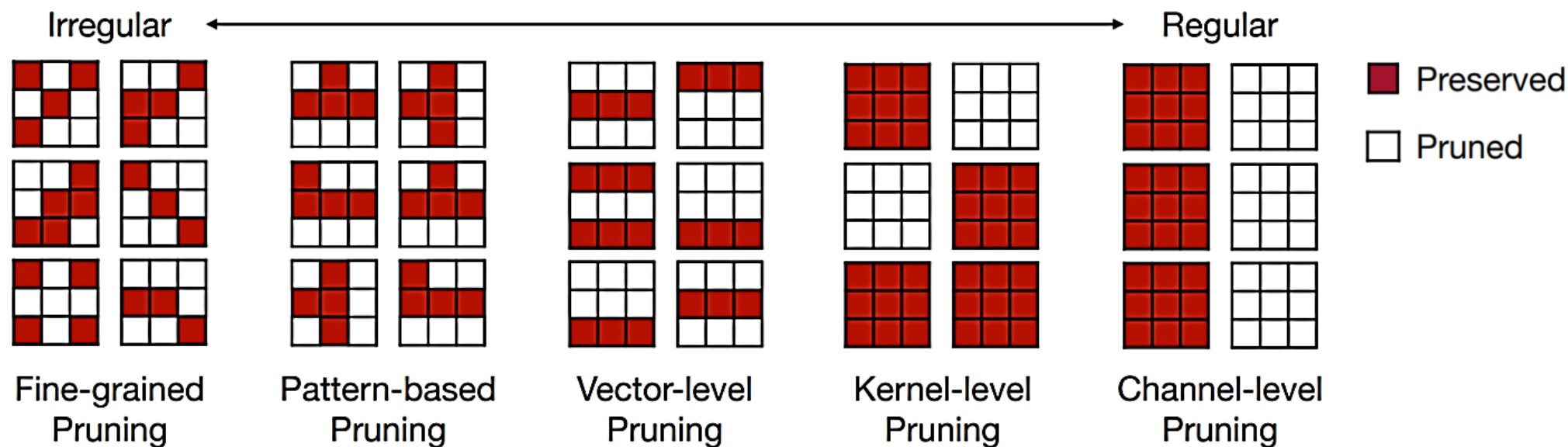
- **模型剪枝(Pruning)**
 - 刪除不重要權重神經元
 - 直接指定刪除比例
 - 不同層刪減不同比例
 - 接近輸入層很敏感
 - 接近輸出層不敏感
 - 考慮硬體運算架構
 - 迭代式 (Iterative)剪枝
 - 修剪
 - 訓練
 - 重複



ALEXNET 卷積層和全連結層剪枝舉推論精確度變化

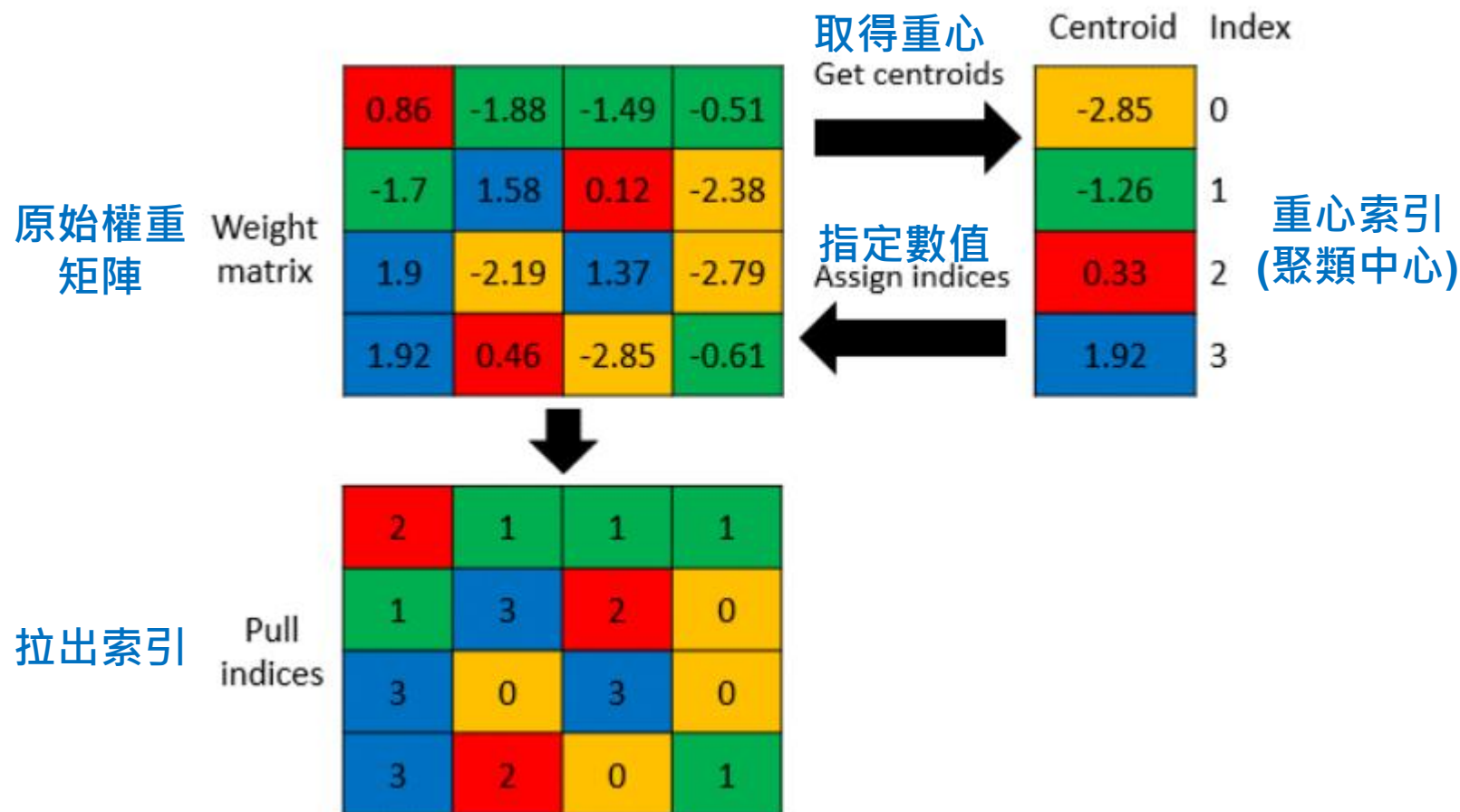
模型剪枝 — 實施方式

- **剪枝方式**：結構化剪枝、非結構化剪枝
- **剪枝粒度**：通道級、內核級、向量級、樣板級、細粒度級
- **剪枝評估**：基於量級、基於梯度、基於冗餘



權重共享

➤ 權重共享(Weight Share / Clustering)



資料來源：<https://blog.tensorflow.org/2020/08/tensorflow-model-optimization-toolkit-weight-clustering-api.html>

矩陣乘法優化 — 標準卷積

➤ 標準2D卷積法(Convolution)

- $c = 1, r = 3, s = 3, m = 5, n = 5$ 時需要
- $r \times s \times m \times n = 225$ 個乘法
- $[(r \times s)-1] \times (m \times n) = 200$ 個加法

c通道
輸入影像

0	0	0	0	0	0	0
0	1	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

r x s卷積核

0	0	1
1	0	0
0	1	1

m x n輸出結果

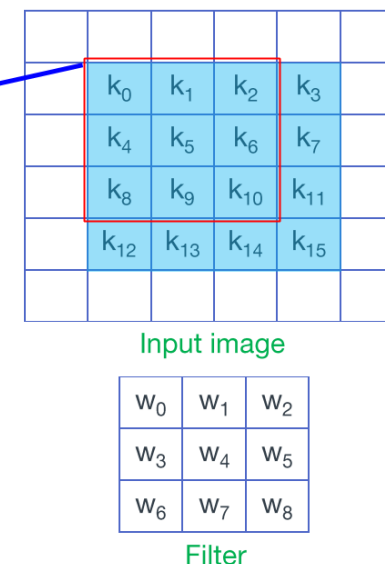
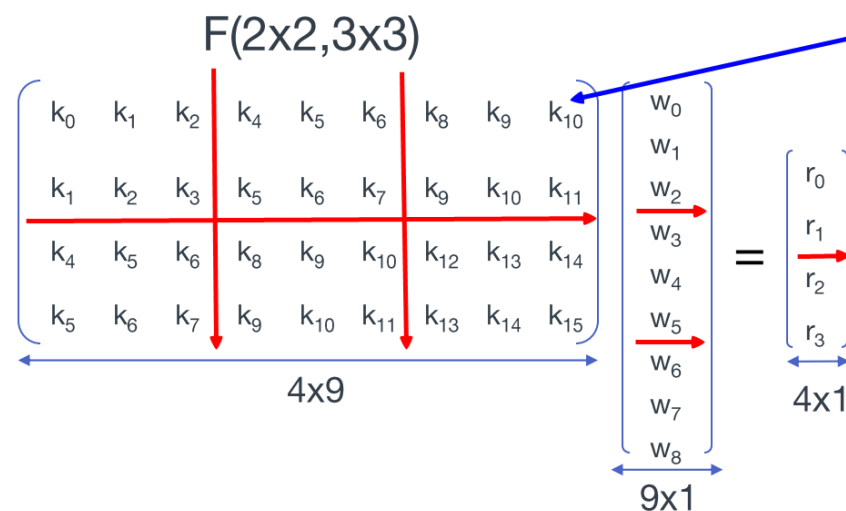
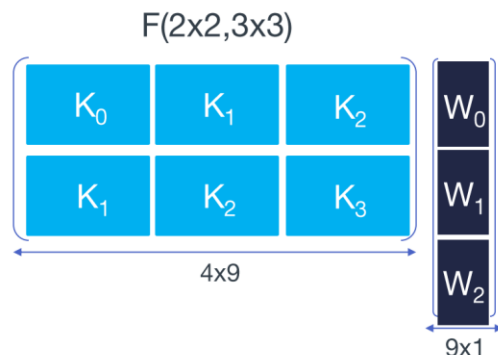
0	1	0	0	0
0	1	1	1	0
1	0	1	2	1
1	4	2	1	0
0	0	1	2	1

矩陣乘法優化 — 快速卷積

快速卷積法(Winograd transformation)

Winograd算法

直接卷積36次乘法
快速卷積16次乘法
減少2.25倍次乘法
可擴展至3維矩陣
適合較小卷積核



$$\begin{bmatrix} K_0 & K_1 & K_2 \\ K_1 & K_2 & K_3 \end{bmatrix} \begin{bmatrix} W_0 \\ W_1 \\ W_2 \end{bmatrix} = \begin{bmatrix} R_0 \\ R_1 \end{bmatrix} = \begin{bmatrix} M_0 + M_1 + M_2 \\ M_1 - M_2 - M_3 \end{bmatrix}$$

Matrix multiply F(2,3)! 4 multiplications

$$M_0 = (K_0 - K_2) \cdot W_0$$

$$M_1 = (K_1 + K_2) \cdot \frac{W_0 + W_1 + W_2}{2}$$

$$M_3 = (K_1 - K_3) \cdot W_2$$

$$M_2 = (K_2 - K_1) \cdot \frac{W_0 - W_1 + W_2}{2}$$

Fast Algorithms for Convolutional Neural Networks <https://arxiv.org/abs/1509.09308>

矩陣乘法優化 — 低秩逼近

➤ 低秩逼近(Low Rank Approximation)

- 去除冗餘資訊，並分解成較小矩陣相乘代替大矩陣相乘。
- 對CNN網路而言
 - 大部份參數來自於全連接層
 - 大部份計算來自於卷積層
 - 卷積時間大部份花費在前幾層
 - 若能有效處理卷積及全連接層可大幅壓縮模型

$$\begin{matrix} & d & & k & & d \\ & \boxed{A} & \approx & \boxed{U} & \times & \boxed{V} \\ v & & & & & \end{matrix}$$

LeNet-5網路結構

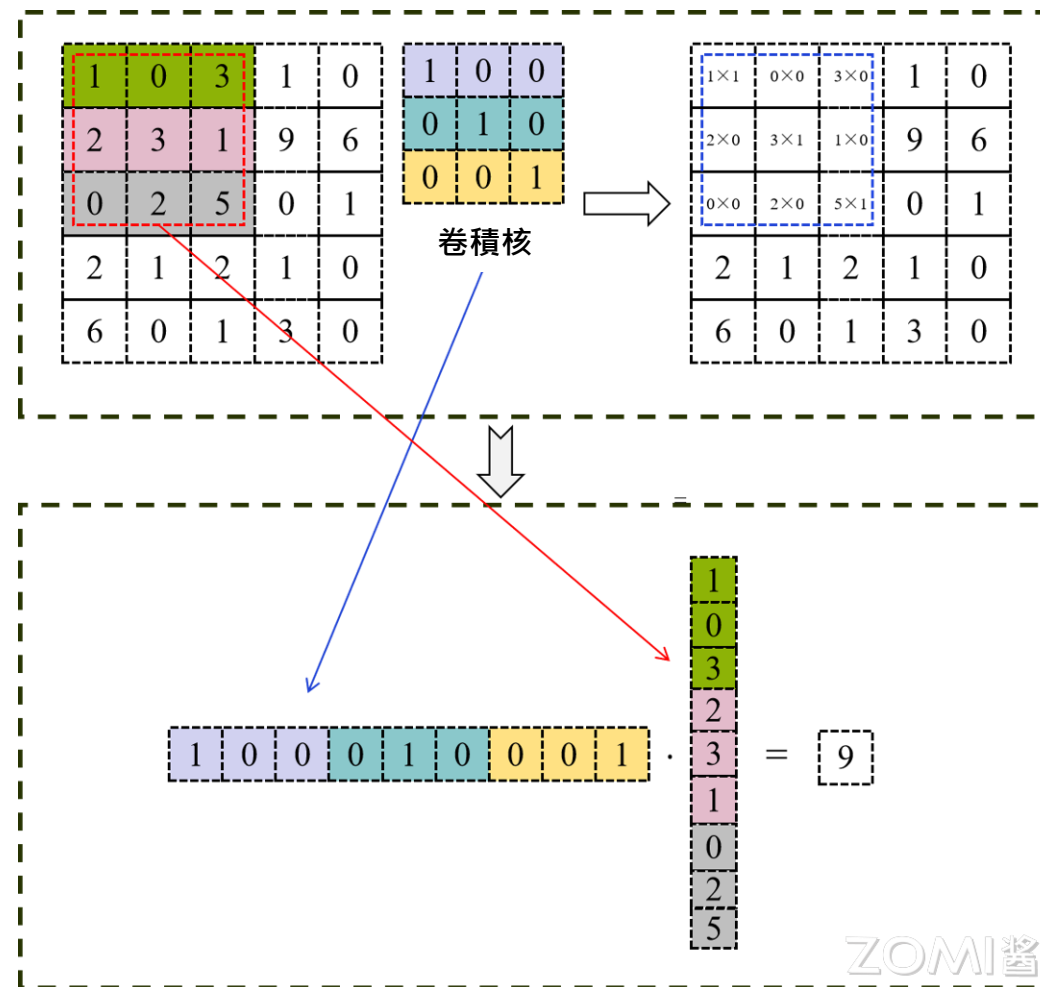
Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully Connected	—	10	—	—	RBF
F6	Fully Connected	—	84	—	—	tanh
C5	Convolution	120	1 × 1	5 × 5	1	tanh
S4	Avg Pooling	16	5 × 5	2 × 2	2	tanh
C3	Convolution	16	10 × 10	5 × 5	1	tanh
S2	Avg Pooling	6	14 × 14	2 × 2	2	tanh
C1	Convolution	6	28 × 28	5 × 5	1	tanh
In	Input	1	32 × 32	—	—	—

矩陣乘法優化 — im2Col

➤ Image to Column

將矩陣展平為一維後再相乘。

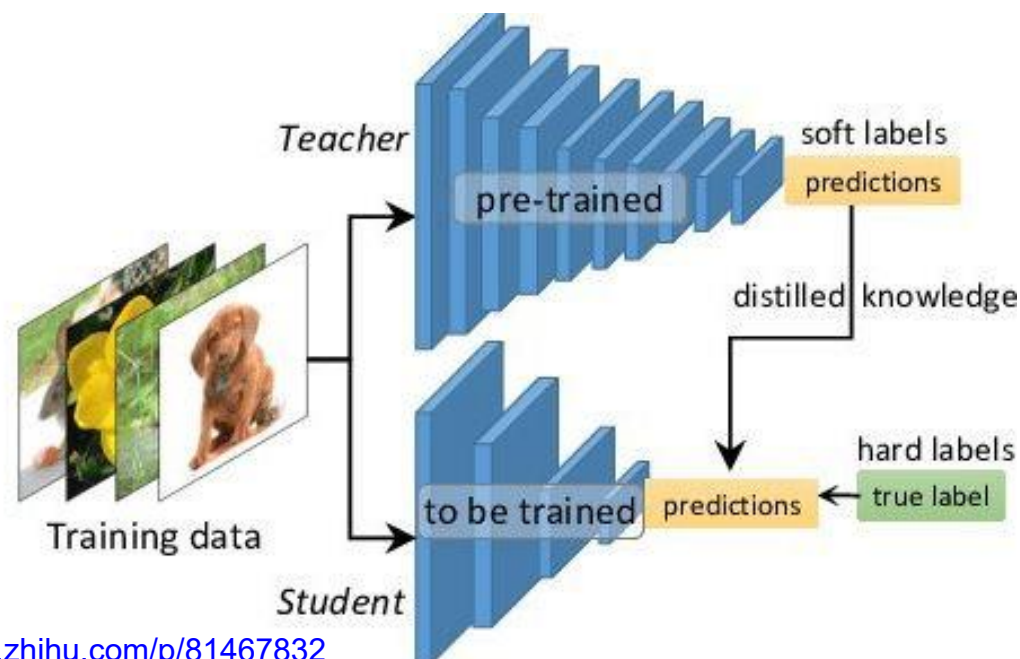
- **優點：**有利矩陣乘法(GEMM)硬體加速計算，包括向量運算指令集，可令資料連續讀取，減少 for 迴圈使用。
- **缺點：**明顯增加計算時記憶體使用量，同一像素被多次複製到新矩陣中。



知識蒸餾

➤ 知識蒸餾(Knowledge Distillation)

- 具有老師和學生網路。老師網路具有複雜結構及高準確度推論能力。學生網路企圖以一簡單網路來學習老師網路輸出結果，如此便將重要資訊保留下來，同時縮小模型，加速推論速度。



資料來源：<https://zhuanlan.zhihu.com/p/81467832>

Intel 神經網路壓縮框架 (NNCF)

Neural Network Compression Framework (NNCF)

原 Intel OpenVINO 工具集之一，
現已獨立，可提供模型訓練後及
訓練期間神經網路壓縮算法。

➤ 訓練後壓縮

- 訓練後量化、權重壓縮、
激活稀疏(僅PyTorch)

➤ 訓練時壓縮(僅PyTorch)

- 量化感知訓練、結合
LoRA 與 NLS 的純權重量
化感知訓練、剪枝

Post-Training Compression Algorithms

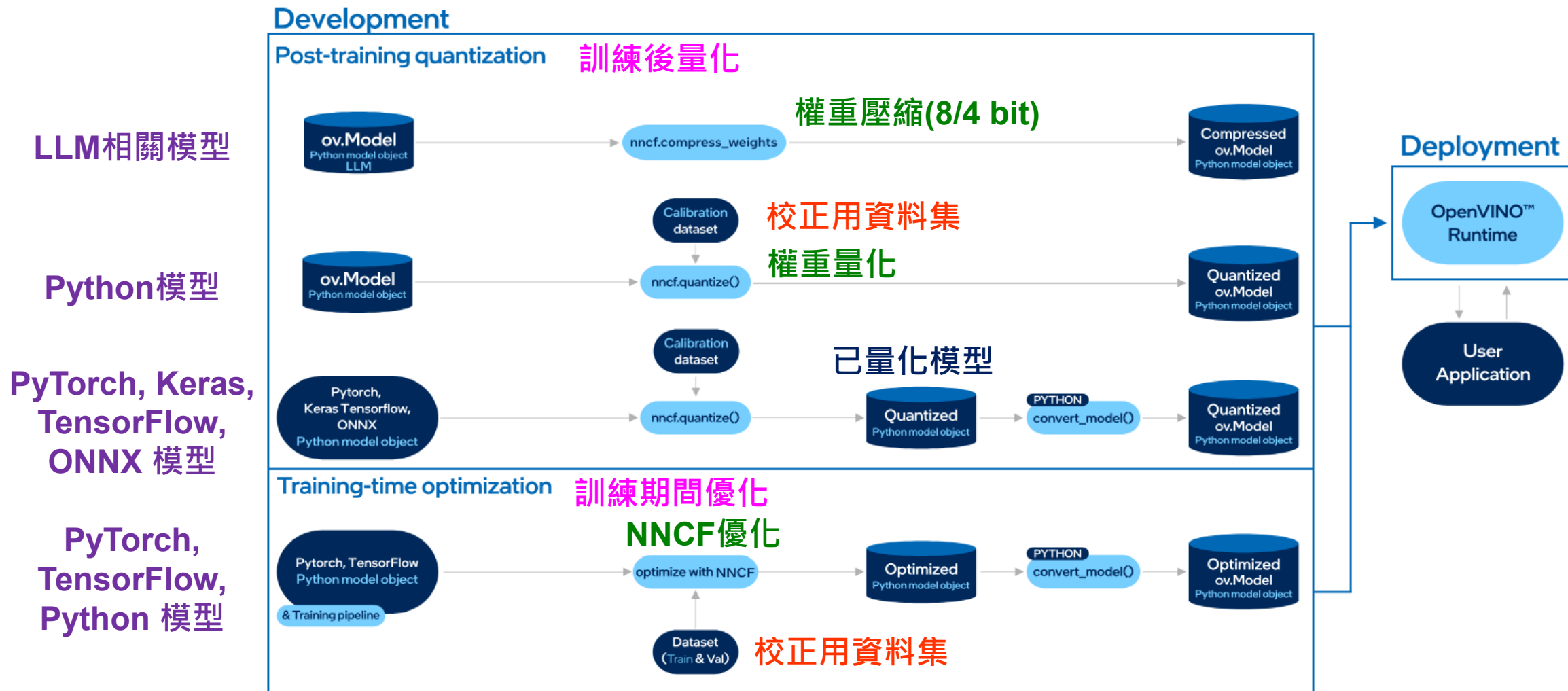
Compression algorithm	OpenVINO	PyTorch	TorchFX	ONNX
Post-Training Quantization	Supported	Supported	Experimental	Supported
Weights Compression	Supported	Supported	Experimental	Supported
Activation Sparsity	Not supported	Experimental	Not supported	Not supported

Training-Time Compression Algorithms

Compression algorithm	PyTorch
Quantization Aware Training	Supported
Weight-Only Quantization Aware Training with LoRA and NLS	Supported
Pruning	Supported

<https://github.com/openvinotoolkit/nncf>

Intel NNCF 工作流程



資料來源：<https://docs.openvino.ai/2026/openvino-workflow/model-optimization.html>

Intel NNCF — 訓練後量化

Post-Training Quantization (PTQ)

只需少量未標註資料作為校驗集

Basic Quantization

`nncf.quantize(model, dataset, 選配參數)`

<https://docs.openvino.ai/2026/openvino-workflow/model-optimization-guide/quantizing-models-post-training/basic-quantization-flow.html>

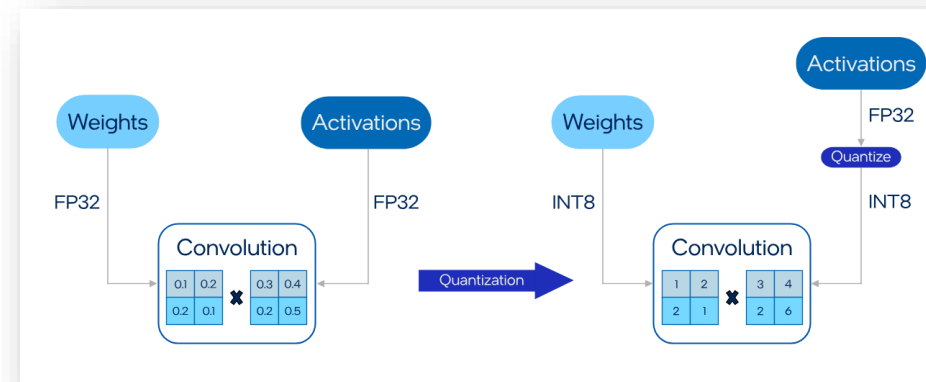
Quantizing with Accuracy Control

可控制推論精度的量化

`nncf.quantize_with_accuracy_control(model, calibration_dataset, valication_dataset, validation_fn, max_drop, drop_type, 選配參數)`

選配參數：model_type, preset, fast_bias_correction, subset_size, ignored_scope, target_device, advanced_parameter

<https://docs.openvino.ai/2026/openvino-workflow/model-optimization-guide/quantizing-models-post-training/quantizing-with-accuracy-control.html>



Intel NNCF — 量化感知訓練

Quantization-Aware Training (QAT)

1. 使用 PTQ 量化模型

2. 精細調校模型

3. 儲存檢查點 (可選)

4. 還原檢查點 (可選)

5. 部署量化模型

```
model = TorchModel() # instance of torch.nn.Module
quantized_model = nncf.quantize(model, ...)
```

```
# tune quantized model for 5 epochs as the baseline
for epoch in range(0, 5):
    for i, data in enumerate(train_loader):
        ... # training loop body
```

```
import openvino as ov

input_fp32 = ... # FP32 model input

# convert PyTorch model to OpenVINO model
ov_quantized_model = ov.convert_model(quantized_model,
example_input=input_fp32)

# compile the model to transform quantized operations to int8
model_int8 = ov.compile_model(ov_quantized_model)

res = model_int8(input_fp32)

# save the model
ov.save_model(ov_quantized_model, "quantized_model.xml")
```

<https://docs.openvino.ai/2026/openvino-workflow/model-optimization-guide/compressing-models-during-training/quantization-aware-training.html>

Intel NNCF — 剪枝

Filter Pruning of Convolutional Models

1. 匯入 NNCF API
2. 建立 NNCF 設定
3. 應用最佳化方法
4. 微調模型
5. 多 GPU 分散式訓練
6. 輸出量化模型
7. 儲存檢查點 (可選)
8. 還原檢查點 (可選)

```
import torch
import nncf # Important - should be imported right after torch
from nncf import NNCFConfig
from nncf.torch import create_compressed_model,
register_default_init_args
```

```
model = TorchModel() # instance of torch.nn.Module
compression_ctrl, model = create_compressed_model(model, nncf_config)
```

```
... # fine-tuning preparations, e.g. dataset, loss, optimization
setup, etc.
```

```
# tune quantized model for 50 epochs as the baseline
for epoch in range(0, 50):
    compression_ctrl.scheduler.epoch_step() # Epoch control API

    for i, data in enumerate(train_loader):
        compression_ctrl.scheduler.step() # Training iteration
        control API
        ... # training loop body
```

```
compression_ctrl.distributed() # call it before the training loop
```

```
compression_ctrl.export_model("compressed_model.onnx")
```

```
checkpoint = {
    'state_dict': model.state_dict(),
    'compression_state': compression_ctrl.get_compression_state(),
    ... # the rest of the user-defined objects to save
}
torch.save(checkpoint, path_to_checkpoint)
```

```
nncf_config_dict = {
    "input_info": {"sample_size": [1, 3, 224, 224]}, # input shape
    required for model tracing
    "compression": [
        {
            "algorithm": "filter_pruning",
            "pruning_init": 0.1,
            "params": {
                "pruning_target": 0.4,
                "pruning_steps": 15
            }
        },
        {
            "algorithm": "quantization", # 8-bit quantization with
            default settings
        }
    ]
}
nncf_config = NNCFConfig.from_dict(nncf_config_dict)
nncf_config = register_default_init_args(nncf_config, train_loader) #
train_loader is an instance of torch.utils.data.DataLoader
```

```
resuming_checkpoint = torch.load(path_to_checkpoint)
compression_state = resuming_checkpoint['compression_state']
compression_ctrl, model = create_compressed_model(model, nncf_config,
compression_state=compression_state)
state_dict = resuming_checkpoint['state_dict']
model.load_state_dict(state_dict)
```

<https://docs.openvino.ai/2026/openvino-workflow/model-optimization-guide/compressing-models-during-training/filter-pruning.html>

參考文獻

- 臺灣科技大學資訊工程系 人工智慧與邊緣運算實務 (CS5149701)
<https://omnixri.blogspot.com/p/ntust-edge-ai.html>
- 陽明交通大學電子工程學系張添烜老師，「深度學習的模型壓縮與加速」(YouTube)
<https://www.youtube.com/playlist?list=PLj6E8qlqmkFv3cCjjX2SA1D4FJ9fadDij>
- MIT Han Lab (韓松)，「TinyML and Efficient Deep Learning Computing | MIT 6.5940 Fall 2024」
<https://hanlab.mit.edu/courses/2024-fall-65940>
- Intel, OpenVINO Document – How to Use - Conventional AI Workflow - Model Optimization – NNCF
<https://docs.openvino.ai/2026/openvino-workflow/model-optimization.html>
- Hamza A. el., Neural Network Quantization for Microcontrollers: A Comprehensive Survey of Methods, Platforms, and Applications
<https://arxiv.org/abs/2508.15008>